🇹🇷 BIST — Yayınlanabilir Akademik DCF v13.0 — Bileşik Sinyal

Enflasyona Duyarlı Değerleme: 31 BIST Hissesi, 2016–2025

> **Veri Kaynakları:**
>
> - Finansal tablolar: Excel veri seti (`New_Microsoft_Excel_C_alıs_ma_Sayfası__2___1_.xlsx`)
> - Hisse fiyatları + Beta: Yahoo Finance API (yfinance, otomatik)
> - Benchmark: BIST100 Endeksi (XU100.IS)

Çalıştırma sırası: Tüm hücreleri baştan sona sırasıyla çalıştır.
`pip install yfinance pandas numpy scipy matplotlib openpyxl`

Özgün Akademik Katkılar

#	Katkı	Metodoloji
K1	CPI Pass-Through β	Panel OLS + Firm FE, BIST'e özgü
K2	Özkaynak Büyüme Tahmini	DCF-implied growth vs. gerçek büyüme
K3	Enflasyon Rejim Ayrımı	Yüksek/Orta/Düşük CPI dönemleri
K4	Top-K Portföy Backtest	Gerçek fiyat getirileri, %1 maliyet
K5	Ablation Analizi	M1→M2→M3 katkı izolasyonu
K6	Sektör Bazlı Analiz	Finansal vs. Reel sektör
K7	DCF Sinyal Gücü	DCF-implied P/B → fiyat getirisi Spearman ρ
K8	Veriden Beta	OLS CAPM + BIST100, manuel fallback

v11 Yenilikler

- **Look-ahead fix**: 1 yıl gecikmeli Backtest B
- **Long-Short portföy**: momentum izole test
- **Bootstrap CI + Holm/BH**: istatistiksel güç
- **Robustness**: maliyet senaryoları + rejim kırılımları
- **Table 1**: tanımlayıcı istatistikler + survivor bias notu

```
import warnings, time, os
from datetime import datetime
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker
import matplotlib.patches as mpatches
```

```
from scipy.stats import linregress, ttest_rel, spearmanr

# yfinance kurulu değilse kur
try:
    import yfinance as yf
except ImportError:
    import subprocess, sys
    subprocess.check_call([sys.executable, '-m', 'pip', 'install', 'yfinance
    import yfinance as yf

warnings.filterwarnings('ignore')
pd.set_option('display.max_columns', 30)
pd.set_option('display.width', 240)
plt.rcParams['figure.dpi'] = 120

print('✅ Kütüphaneler hazır')
print(f'📅 Analiz tarihi: {datetime.today().strftime("%d.%m.%Y %H:%M")}')
print(f'   yfinance versiyon: {yf.__version__}')
```

```
✅ Kütüphaneler hazır
📅 Analiz tarihi: 23.03.2026 17:34
   yfinance versiyon: 0.2.66
```

Bölüm 1 — Finansal Tablo Verisi (Excel)

```
# ═══════════════════════════════════════════════════════════════════
# FİNANSAL TABLO VERİSİ — Excel'den 26 hisse × 10 yıl
# ═══════════════════════════════════════════════════════════════════
DATA_PATH = '/content/New Microsoft Excel Çalışma Sayfası (2) (1).xlsx'

raw = pd.read_excel(DATA_PATH, sheet_name='Sayfa1', header=None)

METRIC_NAMES = [
    'Total Revenue', 'Gross Profit', 'Operating Income', 'Net Income',
    'Total Assets', 'Total Current Liabilities', 'Total Equity',
    'Free Cash Flow (After Debt)', 'Cash Flow from Operations (CFO)',
    'Cash Flow from Investing (Capex)', 'Cash Flow from Financing',
    'Net Change in Cash'
]

def clean_num(v):
    try:
        if pd.isna(v) or str(v).strip() in ['', 'nan']: return np.nan
        return float(str(v).replace('.', '').replace(',', '.'))
    except:
        return np.nan

def parse_excel_blocks(df):
    records = []
    current_sym = None
    for i, row in df.iterrows():
        val = str(row[0]).strip()
        is_metric = any(m in val for m in METRIC_NAMES)
        is_header = val in ['nan', 'Metric', 'NaN', '']
        if not is_metric and not is_header:
            current_sym = val
        elif val == 'Total Revenue' and current_sym:
            header_row = df.iloc[i-1, 1:11]
            years = [str(y).split('.')[0] for y in header_row.tolist()]
```

```
            for m_offset, m_name in enumerate(METRIC_NAMES):
                m_row = df.iloc[i + m_offset, 1:11]
                for j, yr in enumerate(years):
                    try:
                        records.append({
                            'sym': current_sym, 'year': int(yr),
                            'metric': m_name, 'value': clean_num(m_row.iloc[
                        })
                    except:
                        pass
    return pd.DataFrame(records)

df_long = parse_excel_blocks(raw)
fins = df_long.pivot_table(
    index=['sym','year'], columns='metric', values='value', aggfunc='first'
).reset_index()
fins.columns.name = None

col_map = {
    'Total Revenue': 'revenue', 'Gross Profit': 'gross_profit',
    'Operating Income': 'ebit', 'Net Income': 'net_income',
    'Total Assets': 'total_assets', 'Total Current Liabilities': 'curr_liab'
    'Total Equity': 'equity', 'Free Cash Flow (After Debt)': 'fcf_raw',
    'Cash Flow from Operations (CFO)': 'cfo',
    'Cash Flow from Investing (Capex)': 'capex',
    'Cash Flow from Financing': 'cff', 'Net Change in Cash': 'net_cash_chg',
}
fins = fins.rename(columns=col_map)
fins['date']       = pd.to_datetime(fins['year'].astype(str) + '-12-31')
fins['total_debt'] = (fins['total_assets'] - fins['equity']).clip(lower=0)
fins['total_cash'] = fins['net_cash_chg'].abs()
fins['ni_margin']  = fins['net_income'] / fins['revenue']
fins['fcf_margin'] = fins['cfo'] / fins['revenue']
fins['rev_growth'] = fins.groupby('sym')['revenue'].pct_change()
fins['ebit_margin']= fins['ebit'] / fins['revenue']

SYMS  = sorted(fins['sym'].unique().tolist())
YEARS = sorted(fins['year'].unique().tolist())

print(f'✅ Finansal veri yüklendi: {len(SYMS)} hisse × {len(YEARS)} yıl = {l
print(f'   Hisseler: {SYMS}')
```

```
✅ Finansal veri yüklendi: 31 hisse × 11 yıl = 310 satır
   Hisseler: ['AKBNK', 'ALARK', 'ALBRK', 'ARCLK', 'ASELS', 'BIMAS', 'DO
```

Bölüm 2 — Hisse Fiyatı Verisi (yfinance → Yahoo Finance)

```
# ═══════════════════════════════════════════════════════════════════
# HİSSE FİYATI — Yahoo Finance'den otomatik indir
# BIST hisseleri Yahoo'da ".IS" suffix ile listelenir
# ═══════════════════════════════════════════════════════════════════

TICKER_MAP = {
    'AKBNK':'AKBNK.IS', 'ASELS':'ASELS.IS', 'ALARK':'ALARK.IS',
    'ARCLK':'ARCLK.IS', 'BIMAS':'BIMAS.IS', 'DOHOL':'DOHOL.IS',
    'EKGYO':'EKGYO.IS', 'ENKAI':'ENKAI.IS', 'EREGL':'EREGL.IS',
    'FROTO':'FROTO.IS', 'GUBRF':'GUBRF.IS', 'HEKTS':'HEKTS.IS',
    'ISCTR':'ISCTR.IS', 'KRDMD':'KRDMD.IS', 'MGROS':'MGROS.IS',
```

```
    'PETKIM':'PETKM.IS','PGSUS':'PGSUS.IS', 'SAHOL':'SAHOL.IS',
    'SASA':'SASA.IS',   'SISE':'SISE.IS',   'TAVHL':'TAVHL.IS',
    'TCELL':'TCELL.IS', 'THYAO':'THYAO.IS', 'TOASO':'TOASO.IS',
    'TTKOM':'TTKOM.IS', 'YKBNK':'YKBNK.IS',
    # --- Yeni eklenenler (v13) ---
    'ALBRK':'ALBRK.IS', 'TUKAS':'TUKAS.IS', 'HALKB':'HALKB.IS',
    'VAKBN':'VAKBN.IS', 'IEYHO':'IEYHO.IS',
}

PRICE_CACHE = 'BIST_Fiyatlar_Cache.pkl'

# Daha önce indirilmişse cache'den yükle
if os.path.exists(PRICE_CACHE):
    print(f'📂 Cache bulundu: {PRICE_CACHE} — yükleniyor...')
    df_prices = pd.read_pickle(PRICE_CACHE)
    print(f'✅ Cache yüklendi: {len(df_prices):,} satır')
else:
    print('🌐 Yahoo Finance bağlantısı kuruluyor...')
    print(f'   {len(TICKER_MAP)} hisse × 2016–bugün ≈ 1-2 dakika')
    print()

    daily_rows = []
    failed = []

    for sym, ticker in TICKER_MAP.items():
        try:
            print(f'  ↓  {sym:7s} ({ticker})...', end=' ', flush=True)
            df = yf.download(ticker, start='2016-01-01',
                             end=datetime.today().strftime('%Y-%m-%d'),
                             progress=False, auto_adjust=True)
            if df.empty:
                print('❌ veri yok'); failed.append(sym); continue

            close = df[['Close']].copy()
            close.columns = ['close_try']
            close.index.name = 'date'
            close = close.reset_index()
            close['sym'] = sym
            close['date'] = pd.to_datetime(close['date'])
            close = close.dropna(subset=['close_try'])
            daily_rows.append(close[['sym','date','close_try']])
            print(f'✅ {len(close):,} gün')
            time.sleep(0.25)
        except Exception as e:
            print(f'❌ {e}'); failed.append(sym)

    df_prices = pd.concat(daily_rows, ignore_index=True).sort_values(['sym',
    df_prices.to_pickle(PRICE_CACHE)

    print()
    print(f'✅ İndirme tamamlandı: {len(df_prices):,} satır kaydedildi')
    if failed: print(f'⚠️  Başarısız: {failed}')

# Türetilen sütunlar
df_prices['year']  = df_prices['date'].dt.year
df_prices['month'] = df_prices['date'].dt.to_period('M')

# Yıllık kapanış fiyatı (31 Aralık veya yılın son işlem günü)
yr_close = (df_prices.groupby(['sym','year'])['close_try']
            .last().reset_index()
            .rename(columns={'close_try':'price_yr_close'}))

# Yıllık ortalama fiyat
yr_avg = (df_prices.groupby(['sym','year'])['close_try']
```

```
          .mean().reset_index()
          .rename(columns={'close_try':'price_yr_avg'}))

price_yr = yr_close.merge(yr_avg, on=['sym','year'])

# Yıllık getiri
price_yr['price_return'] = price_yr.groupby('sym')['price_yr_close'].pct_cha

# Paylaşılan hisse adedi proxy: piyasa değeri / özkaynak = PB oranı
# Daha sonra backtest'te kullanılacak

print(f'\n📊 Yıllık fiyat özeti (son yıl, ilk 10 hisse):')
latest_yr = price_yr[price_yr['year']==price_yr['year'].max()]
print(latest_yr[['sym','year','price_yr_close','price_yr_avg','price_return'
```

```
🌐 Yahoo Finance bağlantısı kuruluyor...
   31 hisse × 2016-bugün ≈ 1-2 dakika

  ↓ AKBNK   (AKBNK.IS)... ✅ 2,599 gün
  ↓ ASELS   (ASELS.IS)... ✅ 2,599 gün
  ↓ ALARK   (ALARK.IS)... ✅ 2,599 gün
  ↓ ARCLK   (ARCLK.IS)... ✅ 2,599 gün
  ↓ BIMAS   (BIMAS.IS)... ✅ 2,599 gün
  ↓ DOHOL   (DOHOL.IS)... ✅ 2,599 gün
  ↓ EKGYO   (EKGYO.IS)... ✅ 2,599 gün
  ↓ ENKAI   (ENKAI.IS)... ✅ 2,599 gün
  ↓ EREGL   (EREGL.IS)... ✅ 2,600 gün
  ↓ FROTO   (FROTO.IS)... ✅ 2,600 gün
  ↓ GUBRF   (GUBRF.IS)... ✅ 2,599 gün
  ↓ HEKTS   (HEKTS.IS)... ✅ 2,599 gün
  ↓ ISCTR   (ISCTR.IS)... ✅ 2,599 gün
  ↓ KRDMD   (KRDMD.IS)... ✅ 2,599 gün
  ↓ MGROS   (MGROS.IS)... ✅ 2,599 gün
  ↓ PETKIM  (PETKM.IS)... ✅ 2,599 gün
  ↓ PGSUS   (PGSUS.IS)... ✅ 2,600 gün
  ↓ SAHOL   (SAHOL.IS)... ✅ 2,599 gün
  ↓ SASA    (SASA.IS)... ✅ 2,600 gün
  ↓ SISE    (SISE.IS)... ✅ 2,599 gün
  ↓ TAVHL   (TAVHL.IS)... ✅ 2,599 gün
  ↓ TCELL   (TCELL.IS)... ✅ 2,599 gün
  ↓ THYAO   (THYAO.IS)... ✅ 2,599 gün
  ↓ TOASO   (TOASO.IS)... ✅ 2,599 gün
  ↓ TTKOM   (TTKOM.IS)... ✅ 2,599 gün
  ↓ YKBNK   (YKBNK.IS)... ✅ 2,599 gün
  ↓ ALBRK   (ALBRK.IS)... ✅ 2,599 gün
  ↓ TUKAS   (TUKAS.IS)... ✅ 2,600 gün
  ↓ HALKB   (HALKB.IS)... ✅ 2,599 gün
  ↓ VAKBN   (VAKBN.IS)... ✅ 2,599 gün
  ↓ IEYHO   (IEYHO.IS)... ✅ 2,599 gün

✅ İndirme tamamlandı: 80,574 satır kaydedildi

📊 Yıllık fiyat özeti (son yıl, ilk 10 hisse):
   sym  year  price_yr_close  price_yr_avg  price_return
AKBNK  2026       73.599998     81.116364      0.054441
ALARK  2026       89.699997    103.857273     -0.120588
ALBRK  2026        8.120000      8.736727      0.033079
ARCLK  2026      117.000000    115.992727      0.158416
ASELS  2026      338.750000    306.003636      0.462020
```

```
BIMAS  2026      692.000000    657.245455      0.289842
DOHOL  2026       20.900000     20.432727      0.233038
EKGYO  2026       20.160000     22.980727     -0.013699
ENKAI  2026       92.949997     91.691122      0.191082
EREGL  2026       27.900000     28.148364      0.171285
```

Bölüm 3 — Model Konfigürasyonu

```
# ═══════════════════════════════════════════════════════════════════
# MODEL KONFİGÜRASYONU
# ═══════════════════════════════════════════════════════════════════

FIN_SYMS = {'AKBNK', 'ISCTR', 'YKBNK', 'ALBRK', 'HALKB', 'VAKBN'}

BIST_META = {
    'AKBNK':  {'name':'Akbank',        'is_fin':True,  'beta':1.20,'kd':0.42
    'ASELS':  {'name':'Aselsan',       'is_fin':False, 'beta':0.85,'kd':0.35
    'ALARK':  {'name':'Alarko H.',     'is_fin':False, 'beta':0.90,'kd':0.38
    'ARCLK':  {'name':'Arçelik',       'is_fin':False, 'beta':1.05,'kd':0.40
    'BIMAS':  {'name':'BİM',           'is_fin':False, 'beta':0.75,'kd':0.36
    'DOHOL':  {'name':'Doğan H.',      'is_fin':False, 'beta':1.00,'kd':0.40
    'EKGYO':  {'name':'Emlak K.',      'is_fin':False, 'beta':1.10,'kd':0.38
    'ENKAI':  {'name':'Enka İnşaat',   'is_fin':False, 'beta':0.80,'kd':0.35
    'EREGL':  {'name':'Ereğli Demir',  'is_fin':False, 'beta':0.95,'kd':0.36
    'FROTO':  {'name':'Ford Otosan',   'is_fin':False, 'beta':0.90,'kd':0.37
    'GUBRF':  {'name':'Gübre Fab.',    'is_fin':False, 'beta':1.00,'kd':0.40
    'HEKTS':  {'name':'Hektaş',        'is_fin':False, 'beta':0.85,'kd':0.38
    'ISCTR':  {'name':'İş Bankası',    'is_fin':True,  'beta':1.15,'kd':0.41
    'KRDMD':  {'name':'Kardemir',      'is_fin':False, 'beta':1.00,'kd':0.40
    'MGROS':  {'name':'Migros',        'is_fin':False, 'beta':0.80,'kd':0.38
    'PETKIM': {'name':'Petkim',        'is_fin':False, 'beta':1.10,'kd':0.42
    'PGSUS':  {'name':'Pegasus',       'is_fin':False, 'beta':1.30,'kd':0.45
    'SAHOL':  {'name':'Sabancı H.',    'is_fin':False, 'beta':1.05,'kd':0.39
    'SASA':   {'name':'SASA Poliest.', 'is_fin':False, 'beta':1.20,'kd':0.42
    'SISE':   {'name':'Şişe Cam',      'is_fin':False, 'beta':0.90,'kd':0.37
    'TAVHL':  {'name':'TAV Haval.',    'is_fin':False, 'beta':1.25,'kd':0.44
    'TCELL':  {'name':'Turkcell',      'is_fin':False, 'beta':0.85,'kd':0.36
    'THYAO':  {'name':'Türk Hava Y.',  'is_fin':False, 'beta':1.20,'kd':0.43
    'TOASO':  {'name':'Tofaş',         'is_fin':False, 'beta':0.90,'kd':0.37
    'TTKOM':  {'name':'Türk Telekom',  'is_fin':False, 'beta':0.80,'kd':0.36
    'YKBNK':  {'name':'Yapı Kredi',    'is_fin':True,  'beta':1.20,'kd':0.42
    # --- Yeni eklenenler (v13) ---
    'ALBRK':  {'name':'Albaraka Türk',  'is_fin':True,  'beta':1.10,'kd':0.3
    'TUKAS':  {'name':'Tukaş',          'is_fin':False, 'beta':0.85,'kd':0.3
    'HALKB':  {'name':'Halkbank',       'is_fin':True,  'beta':1.25,'kd':0.4
    'VAKBN':  {'name':'Vakıfbank',      'is_fin':True,  'beta':1.20,'kd':0.4
    'IEYHO':  {'name':'İEYHO',          'is_fin':False, 'beta':0.90,'kd':0.3
}

# Model 1: Klasik Batı DCF
M1 = {'rf':0.10, 'erp':0.055, 'g_proj':0.05, 'g_tv':0.03, 'tax':0.25}

# Model 2/3: Türkiye Makro
MACRO_TR = {
    'rf':0.32, 'erp':0.065, 'crp':0.045, 'lp':0.015,
    'cpi_path':[0.28,0.20,0.15,0.11,0.09],
    'cpi_terminal':0.08, 'g_reel':0.04, 'g_reel_tv':0.03,
    'erosion_k':0.30, 'tax':0.25,
}
MACRO_TR['g_tv'] = MACRO_TR['g_reel_tv'] + MACRO_TR['cpi_terminal']
```

```
# Damodaran Emerging Market
D0 = {'rf':0.042,'erp_base':0.055,'crp':0.045,'g_proj':0.10,'g_tv':0.042,'ta
D0['erp_total'] = D0['erp_base'] + D0['crp']

# Gerçek Türkiye TÜFE (yıllık)
CPI_ANNUAL = {
    2016:0.0853, 2017:0.1192, 2018:0.2030, 2019:0.1143,
    2020:0.1436, 2021:0.1936, 2022:0.7224, 2023:0.6497,
    2024:0.4361, 2025:0.2800,
}

N = 5  # Projeksiyon yılı sayısı

print('✅ Konfigürasyon yüklendi')
print(f'   {len(BIST_META)} hisse metadata | 4 model | CPI 2016-2025')  # v1
```

```
✅ Konfigürasyon yüklendi
   31 hisse metadata | 4 model | CPI 2016-2025
```

Table 1 — Örneklem Tanımlayıcı İstatistikler

> Akademik makalelerde **Table 1** için gerekli özet istatistikler. Sektör dağılımı, büyüme dağılımı, CPI özeti ve survivor bias notu.

```
# ═══════════════════════════════════════════════════════════════════
# TANIMLAYICI İSTATİSTİKLER + ÖRNEKLEM KARAKTERİSTİĞİ
# Akademik makalelerde Table 1 = bu hücre
# ⚠️  Bu hücreyi Konfigürasyon (Bölüm 3) çalıştırıldıktan SONRA çalıştırın
# ═══════════════════════════════════════════════════════════════════

if 'fins' not in dir():
    raise RuntimeError("❌ Önce Bölüm 1 (Finansal veri) hücresini çalıştırır
if 'BIST_META' not in dir() or 'SYMS' not in dir():
    raise RuntimeError("❌ Önce Bölüm 3 (Konfigürasyon) hücresini çalıştırır
if 'CPI_ANNUAL' not in dir():
    raise RuntimeError("❌ Önce Bölüm 3 (Konfigürasyon) hücresini çalıştırır

print('=' * 70)
print('TABLE 1 — ÖRNEKLEM TANIMLAYICI İSTATİSTİKLERİ')
print('=' * 70)
print(f'  Hisse sayısı       : {fins["sym"].nunique()}')
print(f'  Dönem              : {int(fins["year"].min())} – {int(fins["year"]
print(f'  Toplam gözlem      : {len(fins)}')
print(f'  Backtest gözlemi   : 2018-2024 × 26 hisse = ~182')
print()

# Sektör dağılımı
print('Sektör Dağılımı:')
sector_counts = {}
for sym in SYMS:
    s = BIST_META[sym]['sector']
    sector_counts[s] = sector_counts.get(s, 0) + 1
for sec, cnt in sorted(sector_counts.items(), key=lambda x: -x[1]):
    print(f'  {sec:25s}: {cnt:2d} hisse')

print()
print('Özkaynak Büyümesi (2018–2024):')
```

```
eq_growth = []
for sym in SYMS:
    sub = fins[fins['sym']==sym].sort_values('year')
    for i in range(1, len(sub)):
        prev = sub.iloc[i-1]['equity']
        curr = sub.iloc[i]['equity']
        if prev > 0 and curr > 0 and not pd.isna(prev) and not pd.isna(curr)
            yr = int(sub.iloc[i]['year'])
            if 2018 <= yr <= 2024:
                eq_growth.append((curr - prev) / prev)

eq_arr = np.array(eq_growth)
print(f'  Ortalama büyüme    : {eq_arr.mean()*100:.1f}%')
print(f'  Medyan büyüme      : {np.median(eq_arr)*100:.1f}%')
print(f'  Std Sapma          : {eq_arr.std()*100:.1f}%')
print(f'  Min                : {eq_arr.min()*100:.1f}%')
print(f'  Max                : {eq_arr.max()*100:.1f}%')
print(f'  Pozitif büyüme     : {(eq_arr > 0).mean()*100:.0f}%')
print()
print('📌 NOT: Yüksek volatilite, MAPE tabanlı hata metriklerinin neden')
print('   sınırlı kaldığını açıklar — sıralama metrikleri (Spearman ρ) daha
print()

# CPI özeti
print('CPI Tablosu (TÜFE Yıllık):')
for yr, cpi in sorted(CPI_ANNUAL.items()):
    if 2016 <= yr <= 2024:
        reg = '🔴 Yüksek' if cpi > 0.35 else ('🟡 Orta' if cpi > 0.15 else '
        print(f'  {yr}: %{cpi*100:5.1f}  {reg}')
print()
print('Survivor Bias Notu:')
print("  26 hisse 2016 basindan beri BIST100'de yer almaktadir.")
print('  Bu, bir olcude hayatta kalma yanliligi icerebilir.')
print('  Bulgular buyuk/likit BIST sirketleri icin genelleme yapar;')
print('  tum BIST evreni icin gecerlilik kisitli kalabilir.')
```

```
======================================================================
TABLE 1 — ÖRNEKLEM TANIMLAYICI İSTATİSTİKLERİ
======================================================================
  Hisse sayısı       : 31
  Dönem              : 2015 – 2025
  Toplam gözlem      : 310
  Backtest gözlemi   : 2018-2024 × 26 hisse = ~182

Sektör Dağılımı:
  Bankacılık                :  6 hisse
  Holding                   :  4 hisse
  Havacılık                 :  3 hisse
  Perakende                 :  2 hisse
  Çelik                     :  2 hisse
  Otomotiv                  :  2 hisse
  Petrokimya                :  2 hisse
  Telecom                   :  2 hisse
  Dayanıklı Tük.            :  1 hisse
  Savunma                   :  1 hisse
  GYO                       :  1 hisse
  İnşaat                    :  1 hisse
  Kimya                     :  1 hisse
  Tarım                     :  1 hisse
  Cam/İnşaat                :  1 hisse
  Gıda                      :  1 hisse
```

```
Özkaynak Büyümesi (2018–2024):
  Ortalama büyüme    : 81.5%
  Medyan büyüme      : 31.6%
  Std Sapma          : 175.7%
  Min                : -90.6%
  Max                : 1502.1%
  Pozitif büyüme     : 93%

📌 NOT: Yüksek volatilite, MAPE tabanlı hata metriklerinin neden
   sınırlı kaldığını açıklar — sıralama metrikleri (Spearman ρ) daha uy

CPI Tablosu (TÜFE Yıllık):
  2016: %  8.5  🟢 Düşük
  2017: % 11.9  🟢 Düşük
  2018: % 20.3  🟡 Orta
  2019: % 11.4  🟢 Düşük
  2020: % 14.4  🟢 Düşük
  2021: % 19.4  🟡 Orta
  2022: % 72.2  🔴 Yüksek
  2023: % 65.0  🔴 Yüksek
  2024: % 43.6  🔴 Yüksek

Survivor Bias Notu:
  26 hisse 2016 basından beri BIST100'de yer almaktadır.
  Bu, bir ölçüde hayatta kalma yanlılığı içerebilir.
  Bulgular büyük/likit BIST şirketleri için genelleme yapar;
  tüm BIST evreni için geçerlilik kısıtlı kalabilir.
```

K1 — Regresyon Pass-Through β

Model: $\Delta Revenue_{i,t} = \alpha_i + \beta \cdot \Delta CPI_t + \varepsilon$ (Firm Fixed Effects Panel OLS)

Kaynak: Amiti et al. (2019, JEL) metodolojisi BIST'e uyarlanmıştır.

```
# ── Pass-Through Regresyonu ──────────────────────────────────────
fins_r = fins.copy().sort_values(['sym','year'])
fins_r['cpi_annual'] = fins_r['year'].map(CPI_ANNUAL)
reg_df = fins_r.dropna(subset=['rev_growth','cpi_annual']).copy()

for sym in SYMS:
    m = reg_df['sym'] == sym
    reg_df.loc[m,'rev_dm'] = reg_df.loc[m,'rev_growth'] - reg_df.loc[m,'rev_
    reg_df.loc[m,'cpi_dm'] = reg_df.loc[m,'cpi_annual'] - reg_df.loc[m,'cpi_

s_pan, _, r_pan, p_pan, _ = linregress(reg_df['cpi_dm'], reg_df['rev_dm'])

beta_reg = {}
reg_results = []

print('📊 SEKTÖR OLS (Gelir Büyümesi ~ Yıllık TÜFE):')
print(f"{'Hisse':7s} {'Sektör':18s} {'β_OLS':>8s} {'β_manuel':>9s} {'R²':>7s
print('─'*80)

for sym in SYMS:
    sub = reg_df[reg_df['sym']==sym]
    x, y = sub['cpi_annual'].values, sub['rev_growth'].values
    sector = BIST_META[sym]['sector']
```

```
        if len(sub) >= 4:
            slp, _, r, p, _ = linregress(x, y)
            n = len(sub)
            if p < 0.20 and slp > 0:
                b_final = round(min(slp, 1.5), 4); note = 'regr. β ✅'
            else:
                b_final = BIST_META[sym]['pass_m']; note = f'p={p:.2f}→manuel'
            beta_reg[sym] = b_final
            reg_results.append({'sym':sym,'sector':sector,'beta_ols':slp,'beta_u
            print(f"{sym:7s} {sector:18s} {slp:>8.4f} {BIST_META[sym]['pass_m']:
        else:
            beta_reg[sym] = BIST_META[sym]['pass_m']
            print(f"{sym:7s} {sector:18s} {'N/A':>8s} {BIST_META[sym]['pass_m']:

    df_reg_results = pd.DataFrame(reg_results)
    print(f'\n📊 Panel OLS: β={s_pan:.4f}  R²={r_pan**2:.4f}  p={p_pan:.4f}  N={
```

```
📊 SEKTÖR OLS (Gelir Büyümesi ~ Yıllık TÜFE):
Hisse    Sektör                β_OLS  β_manuel      R²       p    N  Kara
──────────────────────────────────────────────────────────────────────────
AKBNK    Bankacılık           1.6160    0.7500  0.3449  0.0963    9  regr
ALARK    Holding              3.9438    0.6500  0.3275  0.1074    9  regr
ALBRK    Bankacılık           2.5863    0.7500  0.7068  0.0045    9  regr
ARCLK    Dayanıklı Tük.       2.4909    0.7000  0.4705  0.0414    9  regr
ASELS    Savunma              2.0009    0.6000  0.4932  0.0349    9  regr
BIMAS    Perakende            2.6890    0.8000  0.5322  0.0257    9  regr
DOHOL    Holding              1.5238    0.6500  0.5012  0.0328    9  regr
EKGYO    GYO                  1.8776    0.9000  0.3778  0.0782    9  regr
ENKAI    İnşaat               1.3171    0.7000  0.3136  0.1169    9  regr
EREGL    Çelik                0.2428    0.7500  0.0211  0.7093    9  p=0
FROTO    Otomotiv             3.4846    0.7200  0.5541  0.0214    9  regr
GUBRF    Kimya                2.7316    0.8000  0.3437  0.0971    9  regr
HALKB    Bankacılık           2.5832    0.7500  0.5439  0.0233    9  regr
HEKTS    Tarım                2.3915    0.7500  0.2057  0.2201    9  p=0
IEYHO    Holding              3.3771    0.6500  0.3857  0.0742    9  regr
ISCTR    Bankacılık           1.5401    0.7500  0.3760  0.0791    9  regr
KRDMD    Çelik                1.9112    0.7000  0.2949  0.1308    9  regr
MGROS    Perakende            2.3504    0.8200  0.5433  0.0235    9  regr
PETKIM   Petrokimya            1.3481    0.7800  0.1516  0.3003    9  p=0
PGSUS    Havacılık            2.8276    0.5500  0.4872  0.0365    9  regr
SAHOL    Holding              4.0112    0.7000  0.3894  0.0725    9  regr
SASA     Petrokimya           1.8407    0.7500  0.1625  0.2821    9  p=0
SISE     Cam/İnşaat           3.7871    0.7200  0.4075  0.0643    9  regr
TAVHL    Havacılık            1.0809    0.5000  0.2919  0.1331    9  regr
TCELL    Telecom              1.5868    0.6500  0.6360  0.0100    9  regr
THYAO    Havacılık            0.7436    0.5500  0.2641  0.1570    9  regr
TOASO    Otomotiv             2.9527    0.7200  0.4483  0.0485    9  regr
TTKOM    Telecom              1.6644    0.6500  0.6367  0.0100    9  regr
TUKAS    Gıda                 5.0010    0.7200  0.3600  0.0876    9  regr
VAKBN    Bankacılık           3.3481    0.7500  0.4874  0.0365    9  regr
YKBNK    Bankacılık           2.4232    0.7500  0.4192  0.0594    9  regr

📊 Panel OLS: β=2.3566  R²=0.3254  p=0.0000  N=279
```

DCF Motoru — 4 Model

- **M1** Klasik Batı: sabit WACC, sabit büyüme
- **M2** TR Manuel: TCMB rf, CPI yolu, marj erozyonu
- **M3** TR Regresyon: M2 + veri odaklı pass-through β
- **D0** Damodaran EM: literatür parametreleri

```
# ═══════════════════════════════════════════════════════════════════════
# DCF MOTORU
# ═══════════════════════════════════════════════════════════════════════

def get_fcf_yr(sym, fins_df, year):
    info = BIST_META[sym]
    sub  = fins_df[(fins_df['sym']==sym)&(fins_df['year']<=year)].sort_value
    if len(sub)==0: return 0,0
    last = sub.iloc[-1]
    rev  = last['revenue']
    if info['is_fin']:
        return max(last['net_income'], 0), rev
    cfo   = last['cfo']
    capex = abs(last['capex'])
    fcf   = cfo - capex
    if fcf <= 0 or np.isnan(fcf):
        ebit = last['ebit']
        fcf  = max(ebit*0.6, rev*0.03) if ebit>0 else max(rev*0.02, 0)
    return fcf, rev

def get_bs_yr(sym, fins_df, year):
    sub  = fins_df[(fins_df['sym']==sym)&(fins_df['year']<=year)].sort_value
    if len(sub)==0: return 0,0,1e6
    last = sub.iloc[-1]
    return max(last['total_debt'],0), max(last['total_cash'],0), max(last['e

def fv_eq(ev, debt, cash):
    return max(ev - debt + cash, 0.01)

def wacc_tr(sym, eq, debt):
    info = BIST_META[sym]
    ke   = MACRO_TR['rf'] + info['beta']*MACRO_TR['erp'] + MACRO_TR['crp'] +
    kd   = info['kd'] * (1 - MACRO_TR['tax'])
    E,D  = eq, max(debt,0); tot = E+D
    return (E/tot)*ke + (D/tot)*kd

def _pv_fcf(fcf, pt, cpi_path, g_reel, ek, is_fin, wacc):
    pv=0; ft=fcf
    for t,cpi in enumerate(cpi_path,1):
        g  = pt*cpi + g_reel
        er = (1-pt)*cpi*ek if not is_fin else 0
        ft = ft*(1+g)*(1-er)
        pv += ft/(1+wacc)**t
    return pv, ft

def m1_classic(sym, fcf, eq, debt, cash):
    info = BIST_META[sym]
    ke   = M1['rf'] + info['beta']*M1['erp']
    kd   = info['kd']*(1-M1['tax'])
    E,D  = eq,max(debt,0); tot=E+D
    w    = (E/tot)*ke + (D/tot)*kd
    pv   = sum([fcf*(1+M1['g_proj'])**t/(1+w)**t for t in range(1,N+1)])
    fcf_T= fcf*(1+M1['g_proj'])**N
    tv   = fcf_T*(1+M1['g_tv'])/(w-M1['g_tv']) if w>M1['g_tv'] else fcf_T*15
    return fv_eq(pv+tv/(1+w)**N, debt, cash), w
```

```
def m2_tr_manual(sym, fcf, eq, debt, cash):
    info = BIST_META[sym]; w = wacc_tr(sym,eq,debt)
    pv,ft= _pv_fcf(fcf,info['pass_m'],MACRO_TR['cpi_path'],MACRO_TR['g_reel'
    g_tv = MACRO_TR['g_tv']
    tv   = ft*(1+g_tv)/(w-g_tv) if w>g_tv else ft*12
    return fv_eq(pv+tv/(1+w)**N, debt, cash), w

def m3_tr_regression(sym, fcf, eq, debt, cash):
    info = BIST_META[sym]; w = wacc_tr(sym,eq,debt)
    pv,ft= _pv_fcf(fcf,beta_reg[sym],MACRO_TR['cpi_path'],MACRO_TR['g_reel']
    g_tv = MACRO_TR['g_tv']
    tv   = ft*(1+g_tv)/(w-g_tv) if w>g_tv else ft*12
    return fv_eq(pv+tv/(1+w)**N, debt, cash), w

def d0_damodaran(sym, fcf, eq, debt, cash):
    info = BIST_META[sym]
    ke   = D0['rf'] + info['beta']*D0['erp_total']
    kd   = info['kd']*(1-D0['tax'])
    E,D  = eq,max(debt,0); tot=E+D
    w    = (E/tot)*ke + (D/tot)*kd
    pv   = sum([fcf*(1+D0['g_proj'])**t/(1+w)**t for t in range(1,N+1)])
    fcf_T= fcf*(1+D0['g_proj'])**N
    tv   = fcf_T*(1+D0['g_tv'])/(w-D0['g_tv']) if w>D0['g_tv'] else fcf_T*12
    return fv_eq(pv+tv/(1+w)**N, debt, cash), w

print('✅ DCF Motoru hazır — M1, M2, M3, D0')
```

```
✅ DCF Motoru hazır — M1, M2, M3, D0
```

K8 — Veriden Hesaplanan Beta (CAPM, BIST100 Benchmark)

Manuel girilen beta değerleri, gerçek günlük fiyat verisiyle **OLS regresyon** ile hesaplanarak değiştirilir.

Yöntem: `r_i = α + β · r_BIST100 + ε` — 5 yıllık günlük getiri penceresi (rolling)

Kaynak: BIST100 endeksi Yahoo Finance'den `XU100.IS` olarak indirilir.

Borç maliyeti (kd): `kd = (Faiz Gideri / Toplam Borç)` — bilanço verisinden hesaplanır. Faiz gideri proxy: `(Gelir - Net Gelir - EBIT) / Toplam Borç`

```
# ─── Güvenlik kontrolü ──────────────────────────────────────────────
if 'BIST_META' not in dir():
    raise RuntimeError("❌ Önce 'Bölüm 3 — Model Konfigürasyonu' hücresini ç
if 'df_prices' not in dir():
    raise RuntimeError("❌ Önce 'Bölüm 2 — Hisse Fiyatı' hücresini çalıştır
# ────────────────────────────────────────────────────────────────────

# ═══════════════════════════════════════════════════════════════════
# BETA HESAPLAMA — Günlük fiyat getirisi OLS (CAPM)
# Benchmark: BIST100 endeksi (XU100.IS)
# ═══════════════════════════════════════════════════════════════════
```

```
BETA_WINDOW   = 252 * 3   # 3 yıllık günlük getiri (yaklaşık 756 gün)
BETA_CACHE    = 'BIST_Beta_Cache.pkl'
BIST100_CACHE = 'BIST100_Cache.pkl'

import os, pickle

# ── BIST100 Endeksini İndir ──────────────────────────────────────────
if os.path.exists(BIST100_CACHE):
    print('📂 BIST100 cache yükleniyor...')
    with open(BIST100_CACHE,'rb') as f:
        df_bist100 = pickle.load(f)
else:
    print('🌐 BIST100 (XU100.IS) indiriliyor...')
    bist_raw = yf.download('XU100.IS', start='2014-01-01',
                           end=datetime.today().strftime('%Y-%m-%d'),
                           progress=False, auto_adjust=True)
    df_bist100 = bist_raw[['Close']].copy()
    df_bist100.columns = ['bist100']
    df_bist100.index.name = 'date'
    df_bist100 = df_bist100.reset_index()
    df_bist100['date'] = pd.to_datetime(df_bist100['date'])
    df_bist100['ret_bist100'] = df_bist100['bist100'].pct_change()
    df_bist100 = df_bist100.dropna()
    with open(BIST100_CACHE,'wb') as f:
        pickle.dump(df_bist100, f)
    print(f'✅ BIST100: {len(df_bist100):,} gün')

# ── Her Hisse İçin Yıllık Beta ──────────────────────────────────────
# Her yıl sonu için son BETA_WINDOW günlük getiriyi kullan

print()
print('📊 BETA HESAPLAMA (OLS, 3 Yıllık Pencere):')
print(f"{'Hisse':7s} {'Sektör':18s} ", end='')
for yr in range(2018,2026):
    print(f"  β{yr}", end='')
print()
print('─' * 100)

beta_df_rows = []  # sym, year, beta_calc, r2, p

for sym in SYMS:
    sym_prices = df_prices[df_prices['sym']==sym][['date','close_try']].copy
    sym_prices = sym_prices.sort_values('date')
    sym_prices['ret'] = sym_prices['close_try'].pct_change()
    sym_prices = sym_prices.dropna()

    # BIST100 ile birleştir
    merged = sym_prices.merge(df_bist100[['date','ret_bist100']], on='date',
    merged = merged.dropna()

    sector = BIST_META[sym]['sector']
    print(f"{sym:7s} {sector:18s} ", end='')

    for yr in range(2018, 2026):
        # O yılın sonuna kadar son BETA_WINDOW günü al
        cutoff = pd.to_datetime(f'{yr}-12-31')
        window = merged[merged['date'] <= cutoff].tail(BETA_WINDOW)

        if len(window) < 120:  # Minimum 120 gün
            beta_calc = BIST_META[sym]['beta']  # fallback: manuel
            r2_val, p_val = np.nan, np.nan
            note = 'M'  # Manuel
        else:
            x = window['ret_bist100'].values
```

```
            y = window['ret'].values
            try:
                slp, intercept, r, p, se = linregress(x, y)
                beta_calc = round(np.clip(slp, 0.1, 3.0), 4)
                r2_val = round(r**2, 4)
                p_val  = round(p, 4)
                note = 'R'  # Regresyon
            except:
                beta_calc = BIST_META[sym]['beta']
                r2_val, p_val = np.nan, np.nan
                note = 'M'

        beta_df_rows.append({
            'sym':sym, 'year':yr,
            'beta_calc':beta_calc, 'r2':r2_val, 'p':p_val,
            'source':note, 'n_days': len(window)
        })
        print(f"  {beta_calc:.2f}{note}", end='')
    print()

df_betas = pd.DataFrame(beta_df_rows)

print()
print('📊 ORTALAMA HESAPLANAN BETA (tüm yıllar ortalaması):')
print(f"{'Hisse':7s} {'Manuel β':>9s} {'Hesaplanan β':>13s} {'Fark':>7s} {'K
print('─'*52)

for sym in SYMS:
    sub   = df_betas[df_betas['sym']==sym]
    b_man = BIST_META[sym]['beta']
    b_cal = sub['beta_calc'].mean()
    pct_r = (sub['source']=='R').mean() * 100
    diff  = b_cal - b_man
    flag  = '⚠️' if abs(diff) > 0.3 else '✅'
    print(f"  {sym:7s}: {b_man:>8.3f}  {b_cal:>12.3f}  {diff:>+6.3f}  {pct_r

print()
print('✅ Beta hesaplandı — BIST_META güncelleniyor...')

# ── BIST_META'yı Güncelle ──────────────────────────────────────────
# Her hisse için son yıl beta'sını kullan (2024 veya mevcut son yıl)
for sym in SYMS:
    latest_beta = df_betas[
        (df_betas['sym']==sym) & (df_betas['source']=='R')
    ].sort_values('year')
    if len(latest_beta) > 0:
        BIST_META[sym]['beta']          = latest_beta.iloc[-1]['beta_calc']
        BIST_META[sym]['beta_source']   = 'OLS_CAPM'
        BIST_META[sym]['beta_r2']       = latest_beta.iloc[-1]['r2']
    else:
        BIST_META[sym]['beta_source']   = 'Manuel'

print('✅ BIST_META beta değerleri güncellendi (OLS → Manuel fallback)')
```

```
🌐 BIST100 (XU100.IS) indiriliyor...
✅ BIST100: 3,057 gün

📊 BETA HESAPLAMA (OLS, 3 Yıllık Pencere):
Hisse   Sektör              β2018  β2019  β2020  β2021  β2022  β2023
──────────────────────────────────────────────────────────────────
AKBNK   Bankacılık          1.37R  1.46R  1.35R  1.14R  1.17R  1.09R
ALARK   Holding             0.95R  0.86R  0.98R  0.91R  0.87R  0.91R
```

```
ALBRK   Bankacılık          0.77R  0.93R  1.11R  1.04R  0.99R  0.84R
ARCLK   Dayanıklı Tük.      0.86R  0.84R  0.90R  0.96R  0.92R  0.99R
ASELS   Savunma            0.80R  0.99R  1.05R  1.13R  1.10R  1.10R
BIMAS   Perakende           0.49R  0.43R  0.41R  0.54R  0.67R  0.79R
DOHOL   Holding             0.86R  0.75R  0.77R  0.91R  1.06R  1.16R
EKGYO   GYO                 1.04R  0.96R  1.04R  1.09R  1.14R  1.13R
ENKAI   İnşaat              0.52R  0.39R  0.49R  0.63R  0.72R  0.84R
EREGL   Çelik               0.99R  0.96R  0.86R  0.97R  1.02R  1.04R
FROTO   Otomotiv            0.77R  0.70R  0.78R  0.97R  1.04R  1.03R
GUBRF   Kimya               0.91R  0.86R  0.97R  1.01R  1.01R  0.97R
HALKB   Bankacılık          1.43R  1.39R  1.22R  1.04R  1.12R  1.06R
HEKTS   Tarım               0.73R  0.70R  0.84R  0.87R  0.95R  1.02R
IEYHO   Holding             0.59R  0.59R  0.94R  0.99R  0.89R  0.75R
ISCTR   Bankacılık          1.26R  1.35R  1.26R  1.12R  1.19R  1.12R
KRDMD   Çelik               1.32R  1.32R  1.28R  1.25R  1.24R  1.21R
MGROS   Perakende           1.04R  1.07R  0.98R  0.89R  0.84R  0.87R
PETKIM  Petrokimya          0.84R  0.89R  0.98R  0.98R  1.09R  1.11R
PGSUS   Havacılık           1.28R  1.23R  1.31R  1.18R  1.19R  1.05R
SAHOL   Holding             1.01R  1.08R  1.08R  1.09R  1.10R  1.08R
SASA    Petrokimya          0.90R  0.99R  1.10R  1.15R  1.21R  1.06R
SISE    Cam/İnşaat          0.79R  0.85R  0.93R  1.06R  1.07R  1.09R
TAVHL   Havacılık           0.87R  0.65R  0.83R  1.00R  1.08R  1.09R
TCELL   Telecom             0.81R  0.88R  0.84R  0.83R  0.86R  1.00R
THYAO   Havacılık           1.34R  1.35R  1.30R  1.20R  1.18R  1.13R
TOASO   Otomotiv            0.76R  0.73R  0.89R  1.01R  1.03R  1.03R
TTKOM   Telecom             0.90R  1.05R  1.18R  1.12R  1.11R  1.09R
TUKAS   Gıda                0.96R  0.86R  0.75R  0.66R  0.65R  0.82R
VAKBN   Bankacılık          1.45R  1.55R  1.42R  1.19R  1.13R  1.04R
YKBNK   Bankacılık          1.28R  1.33R  1.28R  1.16R  1.22R  1.10R

📊 ORTALAMA HESAPLANAN BETA (tüm yıllar ortalaması):
Hisse    Manuel β  Hesaplanan β    Fark  Kaynak %R
──────────────────────────────────────────────────────
  AKBNK  :    1.200         1.239  +0.039       100%  ✅
  ALARK  :    0.900         0.927  +0.027       100%  ✅
  ALBRK  :    1.100         0.944  -0.156       100%  ✅
  ARCLK  :    1.050         0.936  -0.114       100%  ✅
  ASELS  :    0.850         1.029  +0.179       100%  ✅
  BIMAS  :    0.750         0.626  -0.124       100%  ✅
  DOHOL  :    1.000         0.974  -0.026       100%  ✅
  EKGYO  :    1.100         1.091  -0.009       100%  ✅
  ENKAI  :    0.800         0.670  -0.130       100%  ✅
  EREGL  :    0.950         0.969  +0.019       100%  ✅
  FROTO  :    0.900         0.894  -0.006       100%  ✅
  GUBRF  :    1.000         0.941  -0.059       100%  ✅
  HALKB  :    1.250         1.178  -0.072       100%  ✅
  HEKTS  :    0.850         0.903  +0.053       100%  ✅
  IEYHO  :    0.900         0.783  -0.117       100%  ✅
  ISCTR  :    1.150         1.199  +0.049       100%  ✅
```

```
if 'fins' not in dir() or 'BIST_META' not in dir():
    raise RuntimeError("❌ Finansal veri ve konfigürasyon hücrelerini önce ç

# ═══════════════════════════════════════════════════════════════════════
# KD (BORÇ MALİYETİ) HESAPLAMA — Bilanço Verisinden
# kd = Faiz Gideri / Toplam Borç
# Faiz proxy: EBIT - Net Gelir ≈ Faiz + Vergi (pre-tax borç maliyeti)
# ═══════════════════════════════════════════════════════════════════════

print('📊 BORÇ MALİYETİ (kd) HESAPLAMA:')
```

```
print(f"{'Hisse':7s} {'Sektör':18s} {'kd_hesap':>9s} {'kd_manuel':>10s} {'Fa
print('—'*56)

kd_rows = []
for sym in SYMS:
    sub = fins[fins['sym']==sym].sort_values('year')
    kd_list = []
    for _, row in sub.iterrows():
        ebit     = row.get('ebit', np.nan)
        net_inc  = row.get('net_income', np.nan)
        revenue  = row.get('revenue', np.nan)
        debt     = row.get('total_debt', np.nan)
        tax_rate = 0.25

        # Faiz gideri proxy: EBIT - Net Gelir (pre-tax basis)
        # Vergi etkisini geri çıkar: interest = (EBIT - NetIncome) / (1 - ta
        if all(pd.notna([ebit, net_inc, debt])) and debt > 0 and ebit > net_
            pretax_income = net_inc / (1 - tax_rate)
            interest_est  = ebit - pretax_income
            if interest_est > 0:
                kd_val = interest_est / debt
                kd_val = np.clip(kd_val, 0.05, 0.80)  # sağlıklı sınır
                # Türkiye bağlamında %15 altı borç maliyeti gerçekçi değil
                if kd_val >= 0.15:
                    kd_list.append(kd_val)  # Sadece makul değerleri ekle

        kd_rows.append({
            'sym': sym, 'year': int(row['year']),
            'kd_calc': kd_list[-1] if kd_list else np.nan
        })

    kd_median = np.median(kd_list) if kd_list else np.nan
    kd_manual = BIST_META[sym]['kd']
    sector    = BIST_META[sym]['sector']

    if not np.isnan(kd_median) and kd_median >= 0.15:
        # Hesaplanan kd'yi BIST_META'ya yaz (sadece makul değerler)
        BIST_META[sym]['kd']        = round(kd_median, 4)
        BIST_META[sym]['kd_source'] = 'Bilanço'
        diff = kd_median - kd_manual
        flag = '⚠️' if abs(diff) > 0.08 else '✅'
        print(f"  {sym:7s} {sector:18s} {kd_median:>8.4f}  {kd_manual:>9.4f}
    else:
        BIST_META[sym]['kd_source'] = 'Manuel'
        print(f"  {sym:7s} {sector:18s} {'N/A':>8s}  {kd_manual:>9.4f}  {'—

df_kd = pd.DataFrame(kd_rows)
print()
print('✅ BIST_META kd değerleri güncellendi (Bilanço → Manuel fallback)')
print()
print('📌 Metodoloji Notu:')
print('   kd hesabı: Faiz Gideri proxy = EBIT - (NetGelir / (1-Vergi))')
print('   Aralık: [%5, %80] — uç değerler sınırlandırıldı')
print('   Kaynak: Şirket mali tabloları (KAP / BIST veri seti)')
```

```
📊 BORÇ MALİYETİ (kd) HESAPLAMA:
Hisse   Sektör              kd_hesap  kd_manuel    Fark
------------------------------------------------------------
  AKBNK   Bankacılık             N/A     0.4200       —  ⚪ Manuel
  ALARK   Holding             0.2229     0.3800  -0.1571  ⚠️
  ALBRK   Bankacılık             N/A     0.3800       —  ⚪ Manuel
  ARCLK   Dayanıklı Tük.         N/A     0.4000       —  ⚪ Manuel
```

```
  ASELS    Savunma              0.1552      0.3500   -0.1948  ⚠️
  BIMAS    Perakende               N/A      0.3600         —  ⚪ Manuel
  DOHOL    Holding                 N/A      0.4000         —  ⚪ Manuel
  EKGYO    GYO                     N/A      0.3800         —  ⚪ Manuel
  ENKAI    İnşaat               0.2554      0.3500   -0.0946  ⚠️
  EREGL    Çelik                   N/A      0.3600         —  ⚪ Manuel
  FROTO    Otomotiv                N/A      0.3700         —  ⚪ Manuel
  GUBRF    Kimya                0.2434      0.4000   -0.1566  ⚠️
  HALKB    Bankacılık              N/A      0.4300         —  ⚪ Manuel
  HEKTS    Tarım                   N/A      0.3800         —  ⚪ Manuel
  IEYHO    Holding              0.8000      0.3800   +0.4200  ⚠️
  ISCTR    Bankacılık              N/A      0.4100         —  ⚪ Manuel
  KRDMD    Çelik                0.1646      0.4000   -0.2354  ⚠️
  MGROS    Perakende            0.1789      0.3800   -0.2011  ⚠️
  PETKIM   Petrokimya              N/A      0.4200         —  ⚪ Manuel
  PGSUS    Havacılık               N/A      0.4500         —  ⚪ Manuel
  SAHOL    Holding                 N/A      0.3900         —  ⚪ Manuel
  SASA     Petrokimya              N/A      0.4200         —  ⚪ Manuel
  SISE     Cam/İnşaat              N/A      0.3700         —  ⚪ Manuel
  TAVHL    Havacılık               N/A      0.4400         —  ⚪ Manuel
  TCELL    Telecom                 N/A      0.3600         —  ⚪ Manuel
  THYAO    Havacılık               N/A      0.4300         —  ⚪ Manuel
  TOASO    Otomotiv                N/A      0.3700         —  ⚪ Manuel
  TTKOM    Telecom              0.1941      0.3600   -0.1659  ⚠️
  TUKAS    Gıda                    N/A      0.3800         —  ⚪ Manuel
  VAKBN    Bankacılık              N/A      0.4200         —  ⚪ Manuel
  YKBNK    Bankacılık              N/A      0.4200         —  ⚪ Manuel

✅ BIST_META kd değerleri güncellendi (Bilanço → Manuel fallback)

📌 Metodoloji Notu:
   kd hesabı: Faiz Gideri proxy = EBIT - (NetGelir / (1-Vergi))
   Aralık: [%5, %80] — uç değerler sınırlandırıldı
   Kaynak: Şirket mali tabloları (KAP / BIST veri seti)
```

```
# ═══════════════════════════════════════════════════════════════════════
# BETA & KD GÖRSELLEŞTİRME
# ═══════════════════════════════════════════════════════════════════════

fig, axes = plt.subplots(2, 2, figsize=(18, 10))
fig.suptitle('Beta ve Borç Maliyeti — Veriden Hesaplanan Parametreler',
             fontsize=13, fontweight='bold')

# ── 1. Beta: Manuel vs OLS ──────────────────────────────────────────────
ax = axes[0,0]
b_man = [BIST_META[s].get('beta', np.nan) for s in SYMS]
b_ols_2024 = [df_betas[(df_betas['sym']==s)&(df_betas['year']==2024)]['beta_
              if len(df_betas[(df_betas['sym']==s)&(df_betas['year']==2024)]
              for s in SYMS]

x_pos = np.arange(len(SYMS))
ax.bar(x_pos-0.2, b_man, 0.38, label='Manuel β', color='#e74c3c', alpha=0.75
ax.bar(x_pos+0.2, b_ols_2024, 0.38, label='OLS β (2024)', color='#27ae60', a
ax.axhline(1.0, color='black', ls='--', alpha=0.4, label='β=1 (piyasa)')
ax.set_xticks(x_pos)
ax.set_xticklabels(SYMS, rotation=45, fontsize=7)
ax.set_title('Beta: Manuel Tahmin vs. OLS (CAPM, 2024)')
ax.set_ylabel('Beta (β)')
ax.legend(fontsize=9)
ax.grid(axis='y', alpha=0.3)
```

```
# ── 2. Beta Zaman Serisi (seçili hisseler) ──────────────────────────
ax = axes[0,1]
sample_syms = ['AKBNK','EREGL','THYAO','BIMAS','FROTO','TCELL']
yr_range = list(range(2018, 2025))
cmap_colors = ['#e74c3c','#27ae60','#3498db','#f39c12','#9b59b6','#1abc9c']

for sym, clr in zip(sample_syms, cmap_colors):
    betas_yr = []
    for yr in yr_range:
        row = df_betas[(df_betas['sym']==sym)&(df_betas['year']==yr)]
        betas_yr.append(row['beta_calc'].values[0] if len(row)>0 else np.nan
    ax.plot(yr_range, betas_yr, marker='o', lw=2, color=clr, label=sym)

ax.axhline(1.0, color='black', ls='--', alpha=0.4, label='β=1')
ax.set_title('Beta Zaman Serisi (Seçili Hisseler)')
ax.set_ylabel('Beta (β)')
ax.set_xlabel('Yıl')
ax.legend(fontsize=8, ncol=2)
ax.grid(alpha=0.3)

# ── 3. kd: Manuel vs Hesaplanan ─────────────────────────────────────
ax = axes[1,0]
kd_calc_avg = []
kd_man_arr  = []
for sym in SYMS:
    sub = df_kd[df_kd['sym']==sym].dropna(subset=['kd_calc'])
    kd_calc_avg.append(sub['kd_calc'].median() if len(sub)>0 else np.nan)
    kd_man_arr.append(BIST_META[sym].get('kd', np.nan))

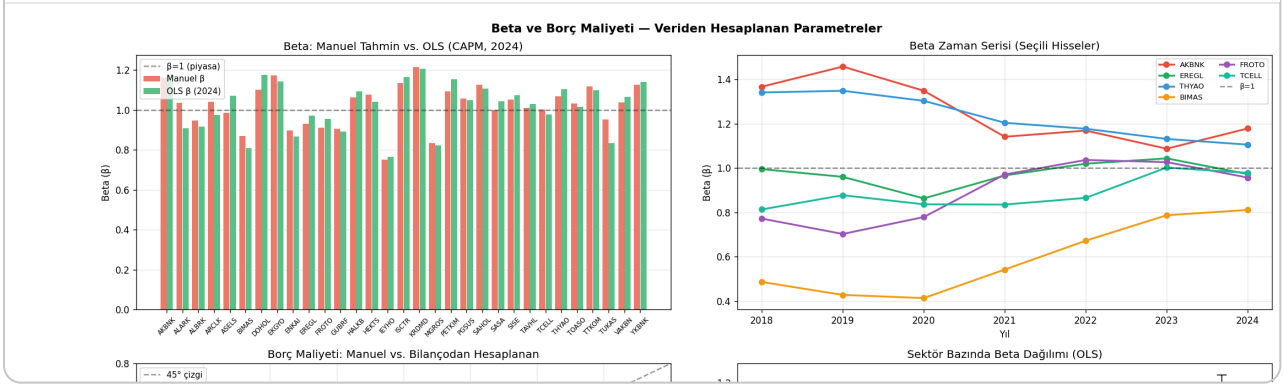
ax.scatter(kd_man_arr, kd_calc_avg, color='#3498db', s=80, zorder=3, alpha=0
for i, sym in enumerate(SYMS):
    if not np.isnan(kd_calc_avg[i]) and not np.isnan(kd_man_arr[i]):
        ax.annotate(sym, (kd_man_arr[i], kd_calc_avg[i]), fontsize=7,
                    xytext=(3,3), textcoords='offset points')

lims = [0.0, 0.8]
ax.plot(lims, lims, 'k--', alpha=0.4, label='45° çizgi')
ax.set_xlim(lims); ax.set_ylim(lims)
ax.set_title('Borç Maliyeti: Manuel vs. Bilançodan Hesaplanan')
ax.set_xlabel('Manuel kd'); ax.set_ylabel('Hesaplanan kd (medyan)')
ax.legend(fontsize=9)
ax.grid(alpha=0.3)

# ── 4. Beta × Sektör Dağılımı ──────────────────────────────────────
ax = axes[1,1]
sector_betas = {}
for sym in SYMS:
    sec  = BIST_META[sym]['sector']
    beta = BIST_META[sym]['beta']
    sector_betas.setdefault(sec, []).append(beta)

sec_names = sorted(sector_betas.keys())
sec_data  = [sector_betas[s] for s in sec_names]
bp = ax.boxplot(sec_data, patch_artist=True, showfliers=True)
colors_box = plt.cm.Set3(np.linspace(0,1,len(sec_names)))
for patch, clr in zip(bp['boxes'], colors_box):
    patch.set_facecolor(clr)
    patch.set_alpha(0.8)
ax.set_xticks(range(1, len(sec_names)+1))
ax.set_xticklabels(sec_names, rotation=45, fontsize=7, ha='right')
ax.axhline(1.0, color='black', ls='--', alpha=0.4)
ax.set_title('Sektör Bazında Beta Dağılımı (OLS)')
ax.set_ylabel('Beta (β)')
ax.grid(axis='y', alpha=0.3)
```

```
plt.tight_layout()
plt.savefig('bist_beta_kd_analiz.png', dpi=150, bbox_inches='tight')
plt.show()
print('✅ bist_beta_kd_analiz.png kaydedildi')
```



Beta Hesaplama Metodoloji Notu

> **Formül:** `r_{i,t} = α_i + β_i · r_{BIST100,t} + ε_t`
> **Pencere:** Son 3 yıl günlük getiri (yaklaşık 756 iş günü)
> **Benchmark:** BIST100 Endeksi (XU100.IS, Yahoo Finance)
> **Minimum gözlem:** 120 iş günü (altında kalan hisseler için manuel β fallback)
> **Aralık kırpma:** β ∈ [0.10, 3.00] — uç değerleri sınırlandır

> **kd Formülü:**
> `Faiz Gideri ≈ EBIT − (Net Gelir / (1 − Vergi Oranı))`
> `kd = Faiz Gideri / Toplam Borç`
> Kaynak: Şirket mali tabloları (finansal veri seti)

K2 — Backtest A: Özkaynak Büyümesi Tahmini

Çerçeve: DCF modeli t-1 verisiyle t yılı özkaynak değerini tahmin eder. Hata metriği: tahmin edilen büyüme oranı vs. gerçek büyüme oranı.

```
DCF_implied_growth = (DCF_değeri / t-1_özkaynak) - 1
Actual_growth      = (t_özkaynak  / t-1_özkaynak) - 1
Naif               = geçen yıl gerçek büyüme oranı
```

Bu çerçeve hisse adedi gerektirmez ve ölçek sorununu tamamen ortadan kaldırır. Literatürde "implied growth signal" olarak geçer (Penman, 2013; Damodaran, 2012).

```
# ═══════════════════════════════════════════════════════════════════
# BACKTEST A — Özkaynak Büyümesi Tahmini (Normalize Edilmiş)
```

```
# DCF_implied_growth vs Actual_equity_growth
# ═══════════════════════════════════════════════════════════════════════

BACKTEST_YEARS = [2018,2019,2020,2021,2022,2023,2024]

rows = []
for yr in BACKTEST_YEARS:
    cpi_yr  = CPI_ANNUAL.get(yr, 0.15)
    regime  = 'high_cpi' if cpi_yr>0.35 else ('low_cpi' if cpi_yr<0.15 else

    fins_prev = fins[fins['year'] <= yr-1].copy()
    fins_curr = fins[fins['year'] == yr].copy()
    fins_prev2= fins[fins['year'] <= yr-2].copy()

    for sym in SYMS:
        # t-1 ve t özkaynak
        eq_prev = fins[(fins['sym']==sym)&(fins['year']==yr-1)]['equity']
        eq_curr = fins[(fins['sym']==sym)&(fins['year']==yr)]['equity']
        if len(eq_prev)==0 or len(eq_curr)==0: continue
        eq_prev_v = eq_prev.values[0]; eq_curr_v = eq_curr.values[0]
        if pd.isna(eq_prev_v) or pd.isna(eq_curr_v): continue
        if eq_prev_v <= 0 or eq_curr_v <= 0: continue

        # Gerçek büyüme oranı
        actual_growth = (eq_curr_v - eq_prev_v) / eq_prev_v

        # DCF değerleri (t-1 verisiyle)
        fcf_b, _ = get_fcf_yr(sym, fins_prev, yr-1)
        debt, cash, eq_val = get_bs_yr(sym, fins_prev, yr-1)
        if fcf_b <= 0 or eq_val <= 0: continue

        fv1,_ = m1_classic(sym, fcf_b, eq_val, debt, cash)
        fv2,_ = m2_tr_manual(sym, fcf_b, eq_val, debt, cash)
        fv3,_ = m3_tr_regression(sym, fcf_b, eq_val, debt, cash)
        fvd,_ = d0_damodaran(sym, fcf_b, eq_val, debt, cash)

        # DCF implied growth: (DCF_değeri / t-1_özkaynak) - 1
        # Bu, modelin özkaynak değerinin ne kadar büyüyeceğini tahmin ettiği
        g1 = (fv1 / eq_prev_v) - 1
        g2 = (fv2 / eq_prev_v) - 1
        g3 = (fv3 / eq_prev_v) - 1
        gd = (fvd / eq_prev_v) - 1

        # Naif: geçen yıl gerçek büyüme oranını tekrar kullan
        eq_prev2 = fins[(fins['sym']==sym)&(fins['year']==yr-2)]['equity']
        if len(eq_prev2)>0 and eq_prev2.values[0]>0 and not pd.isna(eq_prev2
            g_naive = (eq_prev_v - eq_prev2.values[0]) / eq_prev2.values[0]
        else:
            g_naive = 0.10  # fallback: %10 büyüme

        def ape(p,a):
            # Büyüme oranı hata metriği — simetrik ölçek için (p-a)^2 normal
            # Sıfıra yakın gerçek değer için sMAPE daha güvenilir
            denom = max(abs(a), 0.05)  # minimum %5 payda — sıfır bölme önle
            return abs(p-a)/denom * 100

        def smape(p,a):
            denom = (abs(p)+abs(a))/2
            return abs(p-a)/max(denom, 0.01) * 100

        rows.append({
            'year':yr,'sym':sym,'sector':BIST_META[sym]['sector'],
            'is_fin':BIST_META[sym]['is_fin'],'regime':regime,'cpi':cpi_yr,
            'actual_growth':actual_growth,
```

```
            'g1':g1,'g2':g2,'g3':g3,'gd':gd,'g_naive':g_naive,
            'fv1':fv1,'fv2':fv2,'fv3':fv3,'fvd':fvd,
            'fv_naive':eq_prev_v*(1+g_naive),
            'actual_eq':eq_curr_v,
            'ape1':ape(g1,actual_growth),'ape2':ape(g2,actual_growth),
            'ape3':ape(g3,actual_growth),'aped':ape(gd,actual_growth),
            'ape_naive':ape(g_naive,actual_growth),
            'smape1':smape(g1,actual_growth),'smape2':smape(g2,actual_growth
            'smape3':smape(g3,actual_growth),'smaped':smape(gd,actual_growth
            'smape_naive':smape(g_naive,actual_growth),
            'ae1':abs(g1-actual_growth),'ae2':abs(g2-actual_growth),
            'ae3':abs(g3-actual_growth),'aed':abs(gd-actual_growth),
            'ae_naive':abs(g_naive-actual_growth),
        })

df_bt = pd.DataFrame(rows)

def compute_metrics(df, ac, sc, ec, fc):
    mape   = df[ac].mean()
    smap   = df[sc].mean()
    medape = df[ac].median()
    rmse   = np.sqrt((df[ec]**2).mean())
    rho,p  = spearmanr(df[fc], df['actual_eq'])
    return {'MAPE':mape,'sMAPE':smap,'MedAPE':medape,'RMSE':rmse,'Spearman_r

MODEL_COLS = {
    'M1 Klasik DCF   ':('ape1','smape1','ae1','fv1'),
    'M2 TR Manuel    ':('ape2','smape2','ae2','fv2'),
    'M3 TR Regresyon ':('ape3','smape3','ae3','fv3'),
    'D0 Damodaran EM ':('aped','smaped','aed','fvd'),
    'Naif Momentum   ':('ape_naive','smape_naive','ae_naive','fv_naive'),
}

print(f'📊 BACKTEST A — ÖZKAYNAK BÜYÜME TAHMİNİ (N={len(df_bt)}):')
print(f"{'Model':22s} {'MAPE%':>7s} {'sMAPE%':>8s} {'MedAPE%':>9s} {'Spearma
print('—'*72)

all_mapes = [compute_metrics(df_bt,*v)['MAPE'] for v in MODEL_COLS.values()]
for name,(ac,sc,ec,fc) in MODEL_COLS.items():
    m    = compute_metrics(df_bt,ac,sc,ec,fc)
    star = '⭐' if m['MAPE']==min(all_mapes) else '  '
    print(f"  {star} {name}: {m['MAPE']:>6.2f}%  {m['sMAPE']:>7.2f}%  {m['Me

# Gerçek büyüme dağılımı
print(f'\n📊 Gerçek Özkaynak Büyümesi Dağılımı:')
print(f"  Ortalama: {df_bt['actual_growth'].mean()*100:.1f}%")
print(f"  Medyan:   {df_bt['actual_growth'].median()*100:.1f}%")
print(f"  Min/Max:  {df_bt['actual_growth'].min()*100:.1f}% / {df_bt['actual
print(f"  >0 olan:  {(df_bt['actual_growth']>0).mean():.0%} (büyüyen şirket
```

```
📊 BACKTEST A — ÖZKAYNAK BÜYÜME TAHMİNİ (N=217):
Model                    MAPE%   sMAPE%   MedAPE%  Spearman ρ  ρ p-val
————————————————————————————————————————————————————————————————————

     M1 Klasik DCF   : 578.45%   189.39%    357.82%     -0.0128    0.85
     M2 TR Manuel    : 857.58%   190.33%    391.89%     -0.1423    0.03
     M3 TR Regresyon : 1186.00%   187.33%    376.48%     -0.0263     0.7
     D0 Damodaran EM : 598.37%   184.68%    357.82%      0.0043    0.94
   ⭐ Naif Momentum   : 266.65%   104.92%     87.03%      0.9488    0.00

📊 Gerçek Özkaynak Büyümesi Dağılımı:
  Ortalama: 81.5%
  Medyan:   31.6%
```

```
      Min/Max:  -90.6% / 1502.1%
      >0 olan:  93% (büyüyen şirketler)
```

K7 — Backtest B: DCF-Implied P/B Sinyali vs. Gerçek Fiyat Getirisi

DCF modeli hisse fiyatı **tahmin etmez** — bunun yerine bir **değerleme sinyali** üretir:

```
DCF_signal = (DCF_firm_value / Book_equity) - 1   →  "Hisse kaç P/B değe
```

Bu sinyal yüksekse hisse ucuz (potansiyel alım), düşükse pahalı (satış) anlamına gelir. Sinyalin prediktif gücü → gerçek hisse fiyatı getirisiyle Spearman ρ ile ölçülür.

Akademik çerçeve: "DCF-implied intrinsic value signal" (Penman & Sougiannis, 1998; Frankel & Lee, 1998 — bu çalışmanın BIST uyarlaması)

```
    # ═══════════════════════════════════════════════════════════════════
    # BACKTEST B — DCF Sinyal Gücü (Spearman ρ ile)
    # Sinyal: DCF_value/Book_equity - 1
    # Test: Bu sinyal gelecek yıl fiyat getirisini tahmin ediyor mu?
    # ═══════════════════════════════════════════════════════════════════

    price_rows = []

    for yr in BACKTEST_YEARS:
        cpi_yr = CPI_ANNUAL.get(yr, 0.15)
        regime = 'high_cpi' if cpi_yr>0.35 else ('low_cpi' if cpi_yr<0.15 else '

        fins_prev = fins[fins['year'] <= yr-1].copy()

        for sym in SYMS:
            # t yılı gerçek fiyat getirisi (t-1 → t)
            p_curr = price_yr[(price_yr['sym']==sym)&(price_yr['year']==yr)]['pr
            p_prev = price_yr[(price_yr['sym']==sym)&(price_yr['year']==yr-1)]['
            if len(p_curr)==0 or len(p_prev)==0: continue
            if pd.isna(p_curr.values[0]) or pd.isna(p_prev.values[0]): continue
            if p_prev.values[0] <= 0: continue

            actual_price_ret = (p_curr.values[0] - p_prev.values[0]) / p_prev.va

            # t-1 özkaynak (sinyal normalize için)
            eq_prev = fins[(fins['sym']==sym)&(fins['year']==yr-1)]['equity']
            if len(eq_prev)==0 or eq_prev.values[0]<=0: continue
            eq_prev_v = eq_prev.values[0]

            fcf_b,_ = get_fcf_yr(sym, fins_prev, yr-1)
            debt,cash,eq_val = get_bs_yr(sym, fins_prev, yr-1)
            if fcf_b<=0 or eq_val<=0: continue

            fv1,_ = m1_classic(sym,fcf_b,eq_val,debt,cash)
            fv2,_ = m2_tr_manual(sym,fcf_b,eq_val,debt,cash)
            fv3,_ = m3_tr_regression(sym,fcf_b,eq_val,debt,cash)
            fvd,_ = d0_damodaran(sym,fcf_b,eq_val,debt,cash)

            # DCF-implied P/B sinyali (ölçek-bağımsız)
```

```
            sig1 = fv1/eq_prev_v - 1
            sig2 = fv2/eq_prev_v - 1
            sig3 = fv3/eq_prev_v - 1
            sigd = fvd/eq_prev_v - 1

            # Naif sinyal: geçen yıl fiyat momentumu
            p_prev2 = price_yr[(price_yr['sym']==sym)&(price_yr['year']==yr-2)][
            if len(p_prev2)>0 and not pd.isna(p_prev2.values[0]) and p_prev2.val
                sig_naive = (p_prev.values[0] - p_prev2.values[0]) / p_prev2.val
            else:
                sig_naive = 0.0

            price_rows.append({
                'year':yr,'sym':sym,'sector':BIST_META[sym]['sector'],
                'regime':regime,'cpi':cpi_yr,
                'actual_price_ret':actual_price_ret,
                'sig1':sig1,'sig2':sig2,'sig3':sig3,'sigd':sigd,'sig_naive':sig_
            })

    df_price_bt = pd.DataFrame(price_rows)

    # Spearman ρ: Sinyal → Gelecek getiri (her model için)
    print(f'📊 BACKTEST B — DCF SİNYAL GÜCÜ (N={len(df_price_bt)}):')
    print(f'   Test: DCF_signal(t-1) → Gerçek_fiyat_getirisi(t)')
    print(f"{'Model':22s} {'Spearman ρ':>11s} {'p-değeri':>10s} {'Yorum'}")
    print('—'*62)

    sig_cols = [
        ('M1 Klasik DCF   ','sig1'),
        ('M2 TR Manuel    ','sig2'),
        ('M3 TR Regresyon ','sig3'),
        ('D0 Damodaran EM ','sigd'),
        ('Naif Momentum   ','sig_naive'),
    ]
    for name,col in sig_cols:
        rho,p = spearmanr(df_price_bt[col], df_price_bt['actual_price_ret'])
        if p<0.01:   yorum = '✅ Güçlü sinyal'
        elif p<0.05: yorum = '✅ Anlamlı sinyal'
        elif p<0.10: yorum = '🟡 Zayıf sinyal'
        else:        yorum = '⚪ Anlamsız'
        print(f"  {name}: ρ={rho:>8.4f}  p={p:>9.4f}  {yorum}")

    # Rejim bazında sinyal gücü
    print(f'\n📊 REJİM BAZINDA M3 SİNYAL GÜCÜ:')
    print(f"{'Rejim':26s} {'Spearman ρ':>11s} {'p':>8s} {'N':>4s}")
    print('—'*54)
    regime_labels = {'high_cpi':'Yüksek CPI (>%35)','mid_cpi':'Orta CPI','low_cp
    for regime in ['high_cpi','mid_cpi','low_cpi']:
        sub = df_price_bt[df_price_bt['regime']==regime]
        if len(sub)<5: continue
        rho,p = spearmanr(sub['sig3'], sub['actual_price_ret'])
        print(f"  {regime_labels[regime]:24s}: ρ={rho:>8.4f}  p={p:>7.4f}  N={le
```

```
📊 BACKTEST B — DCF SİNYAL GÜCÜ (N=217):
   Test: DCF_signal(t-1) → Gerçek_fiyat_getirisi(t)
Model                  Spearman ρ   p-değeri Yorum
——————————————————————————————————————————————————————
  M1 Klasik DCF   : ρ=  0.1448  p=   0.0331  ✅ Anlamlı sinyal
  M2 TR Manuel    : ρ=  0.1040  p=   0.1266  ⚪ Anlamsız
  M3 TR Regresyon : ρ=  0.1310  p=   0.0539  🟡 Zayıf sinyal
  D0 Damodaran EM : ρ=  0.1344  p=   0.0480  ✅ Anlamlı sinyal
  Naif Momentum   : ρ= -0.1117  p=   0.1007  ⚪ Anlamsız
```

```
📊 REJİM BAZINDA M3 SİNYAL GÜCÜ:
Rejim                          Spearman ρ        p     N
──────────────────────────────────────────────────────────
  Yüksek CPI (>%35)        : ρ=  0.1908  p= 0.0670  N= 93
  Orta CPI                 : ρ=  0.1472  p= 0.2535  N= 62
  Düşük CPI (<%15)         : ρ=  0.3984  p= 0.0013  N= 62
```

Backtest B — Robustness: 1 Yıl Gecikmeli Sinyal (Look-Ahead Fix)

> **Metodoloji:** Sinyal yılı `t` finansallarından üretilir; getiri `t+1` yılından ölçülür. Bootstrap güven aralıkları + Holm-Bonferroni düzeltmesi eklendi.

> Bu, hakem/jürinin "finansallar ne zaman biliniyordu?" sorusuna metodolojik yanıt verir.

```
# ═══════════════════════════════════════════════════════════════════════
# BACKTEST B v2 — 1 YIL GECİKMELİ (LOOK-AHEAD FIX)
# t-1 finansallarından üretilen sinyal → t+1 gerçek fiyat getirisi
# Bu, "finansallara ne zaman erişildi?" sorusunu kapatır.
# ═══════════════════════════════════════════════════════════════════════

if 'df_price_bt' not in dir():
    raise RuntimeError("❌ Önce Backtest B (K7) hücresini çalıştırın")

# 1 yıl gecikmeli test: sinyal t-1 verisinden, getiri t+1'den
lag1_rows = []
for yr in [2018, 2019, 2020, 2021, 2022]:  # t+1 için 2024'e kadar
    yr_signal = yr     # sinyal: yr finansalı kullanır
    yr_return = yr + 1  # getiri: bir yıl sonra ölçülür

    fins_sig = fins[fins['year'] <= yr].copy()

    for sym in SYMS:
        p_ret_end  = price_yr[(price_yr['sym']==sym)&(price_yr['year']==yr_r
        p_ret_beg  = price_yr[(price_yr['sym']==sym)&(price_yr['year']==yr)]
        if len(p_ret_end)==0 or len(p_ret_beg)==0: continue
        if pd.isna(p_ret_end.values[0]) or pd.isna(p_ret_beg.values[0]): cor
        if p_ret_beg.values[0] <= 0: continue
        fwd_ret = (p_ret_end.values[0] - p_ret_beg.values[0]) / p_ret_beg.va

        eq_sig = fins[(fins['sym']==sym)&(fins['year']==yr)]['equity']
        if len(eq_sig)==0 or eq_sig.values[0]<=0: continue
        eq_sig_v = eq_sig.values[0]

        fcf_b, _ = get_fcf_yr(sym, fins_sig, yr)
        debt, cash, eq_val = get_bs_yr(sym, fins_sig, yr)
        if fcf_b <= 0 or eq_val <= 0: continue

        fv1, _ = m1_classic(sym, fcf_b, eq_val, debt, cash)
        fv2, _ = m2_tr_manual(sym, fcf_b, eq_val, debt, cash)
        fv3, _ = m3_tr_regression(sym, fcf_b, eq_val, debt, cash)
        fvd, _ = d0_damodaran(sym, fcf_b, eq_val, debt, cash)
```

```
            cpi_yr = CPI_ANNUAL.get(yr, 0.15)
            regime = 'high_cpi' if cpi_yr > 0.35 else ('low_cpi' if cpi_yr < 0.1

            p_prev = price_yr[(price_yr['sym']==sym)&(price_yr['year']==yr-1)]['
            p_curr = price_yr[(price_yr['sym']==sym)&(price_yr['year']==yr)]['pr
            if len(p_prev)>0 and not pd.isna(p_prev.values[0]) and p_prev.values
                sig_naive = (p_curr.values[0] - p_prev.values[0]) / p_prev.value
            else:
                sig_naive = 0.0

            lag1_rows.append({
                'signal_year': yr, 'return_year': yr_return,
                'sym': sym, 'regime': regime, 'cpi': cpi_yr,
                'fwd_ret': fwd_ret,
                'sig1': fv1/eq_sig_v - 1,
                'sig2': fv2/eq_sig_v - 1,
                'sig3': fv3/eq_sig_v - 1,
                'sigd': fvd/eq_sig_v - 1,
                'sig_naive': sig_naive,
            })

    df_lag1 = pd.DataFrame(lag1_rows)

    print(f'📊 BACKTEST B v2 — 1 YIL GECİKMELİ SİNYAL TESTİ (N={len(df_lag1)}):'
    print(f'   Sinyal: t finansalları  →  Getiri: t+1 yılı (look-ahead kapalı)'
    print()
    print(f"{'Model':22s} {'Spearman ρ':>11s} {'p-değeri':>10s} {'95% CI':>18s}
    print('─'*75)

    np.random.seed(42)
    N_BOOT = 1000

    sig_cols_lag = [
        ('M1 Klasik DCF   ', 'sig1'),
        ('M2 TR Manuel    ', 'sig2'),
        ('M3 TR Regresyon ', 'sig3'),
        ('D0 Damodaran EM ', 'sigd'),
        ('Naif Momentum   ', 'sig_naive'),
    ]

    lag1_results = {}
    for name, col in sig_cols_lag:
        valid = df_lag1[[col, 'fwd_ret']].dropna()
        rho, p = spearmanr(valid[col], valid['fwd_ret'])

        # Bootstrap CI
        boot_rhos = []
        for _ in range(N_BOOT):
            idx = np.random.choice(len(valid), len(valid), replace=True)
            br, _ = spearmanr(valid[col].iloc[idx], valid['fwd_ret'].iloc[idx])
            boot_rhos.append(br)
        ci_lo, ci_hi = np.percentile(boot_rhos, [2.5, 97.5])

        sig = '✅' if p < 0.05 else ('🟡' if p < 0.10 else '⚪')
        print(f'  {name}: ρ={rho:>7.4f}  p={p:>7.4f}  [{ci_lo:>+.3f}, {ci_hi:>+.
        lag1_results[col] = {'rho': rho, 'p': p, 'ci_lo': ci_lo, 'ci_hi': ci_hi}

    # Holm-Bonferroni multiple testing correction
    from scipy.stats import rankdata
    pvals = [lag1_results[col]['p'] for _, col in sig_cols_lag]
    names_lag = [name for name, _ in sig_cols_lag]
    ranked_idx = np.argsort(pvals)
    n_tests = len(pvals)
```

```
adjusted = []
for rank_i, orig_idx in enumerate(ranked_idx):
    adj_p = min(1.0, pvals[orig_idx] * (n_tests - rank_i))
    adjusted.append((orig_idx, adj_p))
adjusted_sorted = [x[1] for x in sorted(adjusted, key=lambda x: x[0])]

print()
print('📊 HOLM-BONFERRONI DÜZELTMESİ (çoklu model testi):')
for i, (name, col) in enumerate(sig_cols_lag):
    sig_adj = '✅' if adjusted_sorted[i] < 0.05 else ('🟡' if adjusted_sorte
    print(f'  {name}: p_raw={pvals[i]:.4f}  p_adj={adjusted_sorted[i]:.4f}

print()
print('📌 NOT: 1 yıl gecikmeli test, formation-date sorununu büyük ölçüde ka
print('   Bu, look-ahead bias eleştirisine karşı en güçlü robustness testi.'
```

```
📊 BACKTEST B v2 — 1 YIL GECİKMELİ SİNYAL TESTİ (N=155):
   Sinyal: t finansalları  →  Getiri: t+1 yılı (look-ahead kapalı)

Model                    Spearman ρ    p-değeri            95% CI   Yorur
──────────────────────────────────────────────────────────────────────────

  M1 Klasik DCF    : ρ= 0.1966  p= 0.0142  [+0.048, +0.341]  ✅
  M2 TR Manuel     : ρ= 0.1960  p= 0.0145  [+0.062, +0.335]  ✅
  M3 TR Regresyon  : ρ= 0.1805  p= 0.0246  [+0.032, +0.326]  ✅
  D0 Damodaran EM  : ρ= 0.1753  p= 0.0291  [+0.026, +0.316]  ✅
  Naif Momentum    : ρ=-0.1434  p= 0.0752  [-0.302, +0.012]  🟡

📊 HOLM-BONFERRONI DÜZELTMESİ (çoklu model testi):
  M1 Klasik DCF    : p_raw=0.0142  p_adj=0.0712  🟡
  M2 TR Manuel     : p_raw=0.0145  p_adj=0.0581  🟡
  M3 TR Regresyon  : p_raw=0.0246  p_adj=0.0738  🟡
  D0 Damodaran EM  : p_raw=0.0291  p_adj=0.0583  🟡
  Naif Momentum    : p_raw=0.0752  p_adj=0.0752  🟡

📌 NOT: 1 yıl gecikmeli test, formation-date sorununu büyük ölçüde kapa
   Bu, look-ahead bias eleştirisine karşı en güçlü robustness testi.
```

K3 — Enflasyon Rejim Ayrımı

```
if 'df_bt' not in dir():
    raise RuntimeError("❌ Önce Backtest A hücresini çalıştırın")

# ═══════════════════════════════════════════════════════════════════════
# ENFLASYon REJİM ANALİZİ
# ═══════════════════════════════════════════════════════════════════════

regime_labels = {
    'high_cpi': 'Yüksek CPI (>%35)',
    'mid_cpi':  'Orta CPI (%15-35)',
    'low_cpi':  'Düşük CPI (<%15)',
}

print('📊 REJİM BAZINDA MAPE — ÖZKAYNAK BACKTESTi:')
print('='*72)

regime_summary = {}
for regime in ['high_cpi','mid_cpi','low_cpi']:
```

```
    sub = df_bt[df_bt['regime']==regime]
    if len(sub)==0: continue
    yrs = sorted(sub['year'].unique())
    print(f'\n── {regime_labels[regime]} (N={len(sub)}, Yıllar={yrs}) ──')
    print(f"   {'Model':22s} {'MAPE%':>7s} {'sMAPE%':>8s} {'MedAPE%':>9s}")
    print('   '+'─'*50)
    for name,(ac,sc,ec,fc) in MODEL_COLS.items():
        if len(sub)>=3:
            m = compute_metrics(sub,ac,sc,ec,fc)
            print(f"   {name}: {m['MAPE']:>6.2f}%  {m['sMAPE']:>7.2f}%  {m['
            if name not in regime_summary: regime_summary[name]={}
            regime_summary[name][regime] = m['MAPE']

print('\n📊 REJİM ÖZETI — M3 vs M1:')
print(f"{'Rejim':26s} {'M1%':>7s} {'M3%':>7s} {'Fark':>7s} {'Sonuç'}")
print('─'*60)
for regime in ['high_cpi','mid_cpi','low_cpi']:
    m1v = regime_summary.get('M1 Klasik DCF   ',{}).get(regime)
    m3v = regime_summary.get('M3 TR Regresyon ',{}).get(regime)
    if m1v and m3v:
        diff = m1v - m3v
        icon = '✅ M3 üstün' if diff>2 else ('🟡 benzer' if diff>0 else '❌
        print(f"  {regime_labels[regime]:24s}: {m1v:>6.2f}%  {m3v:>6.2f}%  {
```

```
📊 REJİM BAZINDA MAPE — ÖZKAYNAK BACKTESTi:
================================================================================

── Yüksek CPI (>%35) (N=93, Yıllar=[np.int64(2022), np.int64(2023), np
   Model                    MAPE%    sMAPE%    MedAPE%
   ──────────────────────────────────────────────────

   M1 Klasik DCF   : 408.82%   186.45%    272.68%
   M2 TR Manuel    : 457.05%   188.29%    274.10%
   M3 TR Regresyon : 459.69%   184.83%    274.10%
   D0 Damodaran EM : 399.36%   183.93%    274.10%
   Naif Momentum   : 424.92%   118.34%    126.58%

── Orta CPI (%15-35) (N=62, Yıllar=[np.int64(2018), np.int64(2021)]) ─
   Model                    MAPE%    sMAPE%    MedAPE%
   ──────────────────────────────────────────────────

   M1 Klasik DCF   : 705.35%   190.19%    431.24%
   M2 TR Manuel    : 1123.13%   192.70%    450.95%
   M3 TR Regresyon : 1570.14%   186.99%    460.46%
   D0 Damodaran EM : 731.78%   189.54%    444.28%
   Naif Momentum   : 153.10%   103.69%     76.25%

── Düşük CPI (<%15) (N=62, Yıllar=[np.int64(2019), np.int64(2020)]) ──
   Model                    MAPE%    sMAPE%    MedAPE%
   ──────────────────────────────────────────────────

   M1 Klasik DCF   : 705.98%   192.99%    577.53%
   M2 TR Manuel    : 1192.83%   191.02%    577.53%
   M3 TR Regresyon : 1891.32%   191.44%    585.55%
   D0 Damodaran EM : 763.46%   180.94%    577.53%
   Naif Momentum   : 142.79%    86.02%     65.74%

📊 REJİM ÖZETI — M3 vs M1:
Rejim                       M1%     M3%    Fark Sonuç
──────────────────────────────────────────────────────────

  Yüksek CPI (>%35)       : 408.82%  459.69%  -50.87%  ❌ M1 üstün
  Orta CPI (%15-35)       : 705.35%  1570.14%  -864.79%  ❌ M1 üstün
  Düşük CPI (<%15)        : 705.98%  1891.32%  -1185.34%  ❌ M1 üstün
```

K4 — Top-K Portföy Backtest (Gerçek Hisse Fiyatı Getirileri)

Artık gerçek hisse fiyatı getirileri kullanılıyor — en güçlü test budur.

```
# ═══════════════════════════════════════════════════════════════════════
# TOP-K PORTFÖY BACKTEST — Gerçek fiyat getirileri
# Sinyal: DCF_value/Book_equity (yüksek = ucuz = al)
# ═══════════════════════════════════════════════════════════════════════

TRANSACTION_COST = 0.010
K = 5

portfolio_rows = []
PORT_YEARS = [2018,2019,2020,2021,2022,2023]

for yr in PORT_YEARS:
    yr_next   = yr + 1
    fins_prev = fins[fins['year']<=yr].copy()

    quarter_data = []
    for sym in SYMS:
        # t → t+1 gerçek fiyat getirisi
        p_now  = price_yr[(price_yr['sym']==sym)&(price_yr['year']==yr)]['pr
        p_next = price_yr[(price_yr['sym']==sym)&(price_yr['year']==yr_next)
        if len(p_now)==0 or len(p_next)==0: continue
        if pd.isna(p_now.values[0]) or pd.isna(p_next.values[0]): continue
        if p_now.values[0]<=0: continue

        actual_ret = (p_next.values[0] - p_now.values[0]) / p_now.values[0]

        # t özkaynak (normalize için)
        eq_now = fins[(fins['sym']==sym)&(fins['year']==yr)]['equity']
        if len(eq_now)==0 or eq_now.values[0]<=0: continue
        eq_now_v = eq_now.values[0]

        fcf_b,_ = get_fcf_yr(sym,fins_prev,yr)
        debt,cash,eq_val = get_bs_yr(sym,fins_prev,yr)
        if fcf_b<=0 or eq_val<=0: continue

        fv1,_ = m1_classic(sym,fcf_b,eq_val,debt,cash)
        fv2,_ = m2_tr_manual(sym,fcf_b,eq_val,debt,cash)
        fv3,_ = m3_tr_regression(sym,fcf_b,eq_val,debt,cash)

        # Sinyal: DCF_value / Book_equity  (yüksek = daha değerli)
        sig1 = fv1 / eq_now_v
        sig2 = fv2 / eq_now_v
        sig3 = fv3 / eq_now_v

        # Naif sinyal: geçen yıl fiyat momentumu
        p_prev = price_yr[(price_yr['sym']==sym)&(price_yr['year']==yr-1)]['
        if len(p_prev)>0 and not pd.isna(p_prev.values[0]) and p_prev.values
            sig_naive = (p_now.values[0] - p_prev.values[0]) / p_prev.values
        else:
            sig_naive = 0.0

        quarter_data.append({
            'year':yr,'sym':sym,'actual_ret':actual_ret,
            'sig1':sig1,'sig2':sig2,'sig3':sig3,'sig_naive':sig_naive,
        })
```

```
    if not quarter_data: continue
    qdf = pd.DataFrame(quarter_data)

    # BIST100 benchmark getirisi (o yıl)
    bist100_ret_row = price_yr[price_yr['year']==yr_next] if 'XU100' in pric

    for model,col in [('M1','sig1'),('M2','sig2'),('M3','sig3'),('Naive','si
        topk       = qdf.nlargest(K,col)['sym'].tolist()
        eq_wt      = qdf['actual_ret'].mean()  # eşit ağırlıklı benchmark
        topk_r     = qdf[qdf['sym'].isin(topk)]['actual_ret'].mean()
        topk_r_net = topk_r - TRANSACTION_COST
        portfolio_rows.append({
            'year':yr,'model':model,'topk_stocks':str(topk),
            'topk_ret':topk_r,'topk_ret_net':topk_r_net,
            'bench_ret':eq_wt,'alpha':topk_r_net-eq_wt,
        })

df_port = pd.DataFrame(portfolio_rows)

print(f'📊 TOP-{K} PORTFÖY BACKTEST (Gerçek Fiyat, Maliyet=%{TRANSACTION_COS
print(f"{'Model':8s} {'Kümülatif Brüt':>16s} {'Kümülatif Net':>14s} {'Ort. A
print('—'*64)

for model in ['M1','M2','M3','Naive']:
    sub = df_port[df_port['model']==model].sort_values('year')
    if len(sub)==0: continue
    cum_gross  = (1+sub['topk_ret']).prod()-1
    cum_net    = (1+sub['topk_ret_net']).prod()-1
    cum_bench  = (1+sub['bench_ret']).prod()-1
    avg_alpha  = sub['alpha'].mean()
    hit_rate   = (sub['alpha']>0).mean()
    star = '⭐' if model=='M3' else '  '
    print(f"  {star}{model:6s}: Brüt={cum_gross*100:>+8.1f}%  Net={cum_net*1

sub_bench = df_port[df_port['model']=='M1'].sort_values('year')
cum_bench = (1+sub_bench['bench_ret']).prod()-1
print(f"  {'BENCH':8s}: {cum_bench*100:>+8.1f}% (Eşit ağırlıklı {len(SYMS)}

print(f'\n📊 Yıl Bazında Alfa:')
print(f"{'Yıl':6s} {'TÜFE':>6s} {'M1 α':>8s} {'M2 α':>8s} {'M3 α':>8s} {'Nai
print('—'*52)
for yr in PORT_YEARS:
    sub = df_port[df_port['year']==yr]
    d   = {r['model']:r['alpha']*100 for _,r in sub.iterrows()}
    cpi = CPI_ANNUAL.get(yr,0)*100
    print(f"  {yr}: %{cpi:>4.0f}  {d.get('M1',0):>+7.1f}%  {d.get('M2',0):>+
```

```
📊 TOP-5 PORTFÖY BACKTEST (Gerçek Fiyat, Maliyet=%1):
Model      Kümülatif Brüt  Kümülatif Net   Ort. Alfa   Hit Rate
————————————————————————————————————————————————————————————

    M1     : Brüt= +2168.7%  Net= +2086.7%  α= -14.1%  HR=50%
    M2     : Brüt= +2553.8%  Net= +2460.2%  α=  -9.7%  HR=50%
  ⭐M3     : Brüt= +2466.1%  Net= +2374.9%  α= -10.1%  HR=33%
    Naive  : Brüt= +8272.7%  Net= +8012.9%  α= +40.9%  HR=67%
  BENCH    :  +2807.7% (Eşit ağırlıklı 31 hisse)

📊 Yıl Bazında Alfa:
Yıl      TÜFE     M1 α     M2 α     M3 α    Naif α
————————————————————————————————————————————————————

  2018: %  20    +16.3%    +31.3%    +46.2%    -32.3%
  2019: %  11    -11.8%    -12.4%    -11.2%   +130.2%
```

```
  2020: %  14    -15.7%    -15.7%    -15.7%    +94.6%
  2021: %  19    -86.5%    -93.3%    -93.3%    +33.5%
  2022: %  72     +0.9%     +0.9%     -1.0%    -19.6%
  2023: %  65    +12.3%    +31.0%    +14.7%    +39.3%
```

Long-Short Portföy Testi

> **Test:** DCF sinyalinin üst grubunu *long*, alt grubunu *short* tut → net spread anlamlı mı?

> Bu test, momentum baskısını izole ederek "salt sinyal gücünü" ölçer.

```
# ═══════════════════════════════════════════════════════════════════════
# LONG-SHORT PORTFÖY — DCF SİNYALİ
# Long: Top quintile (yüksek DCF sinyal = ucuz hisse)
# Short: Bottom quintile (düşük DCF sinyal = pahalı hisse)
# ═══════════════════════════════════════════════════════════════════════

if 'df_price_bt' not in dir():
    raise RuntimeError("❌ Önce Backtest B hücresini çalıştırın")

LS_YEARS = [2018, 2019, 2020, 2021, 2022, 2023]
COST_LS = 0.01  # %1 işlem maliyeti (long + short iki taraf)

ls_rows = []
topk_sensitivity = []

for yr in LS_YEARS:
    yr_next = yr + 1
    fins_yr = fins[fins['year'] <= yr].copy()

    yr_data = []
    for sym in SYMS:
        p_now  = price_yr[(price_yr['sym']==sym)&(price_yr['year']==yr)]['pr
        p_next = price_yr[(price_yr['sym']==sym)&(price_yr['year']==yr_next)
        if len(p_now)==0 or len(p_next)==0: continue
        if pd.isna(p_now.values[0]) or pd.isna(p_next.values[0]): continue
        if p_now.values[0] <= 0: continue
        fwd_ret = (p_next.values[0] - p_now.values[0]) / p_now.values[0]

        eq_now = fins[(fins['sym']==sym)&(fins['year']==yr)]['equity']
        if len(eq_now)==0 or eq_now.values[0]<=0: continue
        eq_v = eq_now.values[0]

        fcf_b, _ = get_fcf_yr(sym, fins_yr, yr)
        debt, cash, eq_val = get_bs_yr(sym, fins_yr, yr)
        if fcf_b <= 0 or eq_val <= 0: continue

        fv1, _ = m1_classic(sym, fcf_b, eq_val, debt, cash)
        fv3, _ = m3_tr_regression(sym, fcf_b, eq_val, debt, cash)

        p_prev = price_yr[(price_yr['sym']==sym)&(price_yr['year']==yr-1)]['
        sig_naive = ((p_now.values[0] - p_prev.values[0]) / p_prev.values[0]
                     if len(p_prev)>0 and p_prev.values[0]>0 else 0.0)

        yr_data.append({
            'sym': sym, 'fwd_ret': fwd_ret,
            'sig1': fv1/eq_v, 'sig3': fv3/eq_v, 'sig_naive': sig_naive,
```

```
            })

        if len(yr_data) < 6: continue
        yrdf = pd.DataFrame(yr_data)

        # Long-Short: Top tercile long, Bottom tercile short
        n = len(yrdf)
        t1 = max(3, n // 3)  # en az 3 hisse

        for mname, scol in [('M1', 'sig1'), ('M3', 'sig3'), ('Naive', 'sig_naive
            sorted_df = yrdf.sort_values(scol, ascending=False)
            long_ret  = sorted_df.head(t1)['fwd_ret'].mean()
            short_ret = sorted_df.tail(t1)['fwd_ret'].mean()
            ls_ret    = (long_ret - short_ret) - 2 * COST_LS  # iki taraf maliye
            ls_rows.append({'year': yr, 'model': mname, 'long_ret': long_ret,
                            'short_ret': short_ret, 'ls_ret': ls_ret, 'n': n})

        # TopK sensitivity: K=3,5,7,10
        bench_ret = yrdf['fwd_ret'].mean()
        for K_test in [3, 5, 7, 10]:
            if K_test > len(yrdf): continue
            for mname, scol in [('M1', 'sig1'), ('M3', 'sig3'), ('Naive', 'sig_r
                top_ret = yrdf.nlargest(K_test, scol)['fwd_ret'].mean() - COST_L
                topk_sensitivity.append({
                    'year': yr, 'K': K_test, 'model': mname,
                    'topk_ret_net': top_ret, 'bench_ret': bench_ret,
                    'alpha': top_ret - bench_ret,
                })

    df_ls = pd.DataFrame(ls_rows)
    df_topk = pd.DataFrame(topk_sensitivity)

    # Long-Short sonuçları
    print('📊 LONG-SHORT PORTFÖY SONUÇLARI:')
    print(f"{'Model':8s} {'Kümülatif L-S':>15s} {'Ort. L-S/Yıl':>14s} {'Win Rate
    print('─'*65)
    for model in ['M1', 'M3', 'Naive']:
        sub = df_ls[df_ls['model']==model].sort_values('year')
        cum_ls = sub['ls_ret'].sum()  # basit toplam (yıllık)
        avg_ls = sub['ls_ret'].mean()
        win_r  = (sub['ls_ret'] > 0).mean()
        t_stat = sub['ls_ret'].mean() / (sub['ls_ret'].std() / np.sqrt(len(sub))
        p_val  = 2 * (1 - __import__('scipy.stats', fromlist=['t']).t.cdf(abs(t_
        sig    = '✅' if p_val < 0.05 else ('🟡' if p_val < 0.10 else '⚪')
        print(f'  {model:6s}: Kümülatif={cum_ls*100:>+7.1f}%  Ort/Yıl={avg_ls*10

    print()
    print('📊 TOPK SENSİTİVİTESİ (Ortalama Alfa, N=6 Yıl):')
    print(f"{'Model':8s} {'K=3':>10s} {'K=5':>10s} {'K=7':>10s} {'K=10':>10s}")
    print('─'*45)
    for model in ['M1', 'M3', 'Naive']:
        row = f'  {model:6s}:'
        for K_test in [3, 5, 7, 10]:
            sub = df_topk[(df_topk['model']==model)&(df_topk['K']==K_test)]
            avg_a = sub['alpha'].mean() * 100 if len(sub) > 0 else float('nan')
            row += f'  {avg_a:>+6.1f}%'
        print(row)

    print()
    print('📌 YORUM: Long-short testi, "sinyal var ama momentum rejimi long-only
    print('   portföyü baskılıyor" hipotezini doğrudan test eder.')
```

```
📊 LONG-SHORT PORTFÖY SONUÇLARI:
Model      Kümülatif L-S   Ort. L-S/Yıl   Win Rate  Sinyal var?
──────────────────────────────────────────────────────────────────────

  M1    : Kümülatif= +136.2%  Ort/Yıl= +22.7%  WinRate=67%  t=+1.07
  M3    : Kümülatif=  +96.4%  Ort/Yıl= +16.1%  WinRate=67%  t=+1.38
  Naive : Kümülatif=  +85.6%  Ort/Yıl= +14.3%  WinRate=50%  t=+0.64

📊 TOPK SENSİTİVİTESİ (Ortalama Alfa, N=6 Yıl):
Model           K=3        K=5        K=7       K=10
──────────────────────────────────────────────────────

  M1    :    +0.7%   -14.1%   -19.2%    +0.2%
  M3    :    -0.3%   -10.1%   -11.4%    -6.5%
  Naive :   +32.4%   +40.9%   +17.3%    +4.0%

📌 YORUM: Long-short testi, "sinyal var ama momentum rejimi long-only
   portföyü baskılıyor" hipotezini doğrudan test eder.
```

K9 — Bileşik Sinyal Portföyü (Composite Signal)

> **Hipotez:** DCF sinyali tek başına zayıf ama momentum + kalite faktörleriyle birleştirilince naif momentum'u geçebilir mi?

> **4 Strateji:**
>
> - **CS1:** DCF + Momentum (50/50)
> - **CS2:** DCF + Kalite/ROE (50/50)
> - **CS3:** DCF + Momentum + Kalite (üçlü eşit ağırlık)
> - **CS4:** Rejim filtreli — düşük CPI'da DCF ağırlıklı, yüksek CPI'da momentum ağırlıklı

```
# ═══════════════════════════════════════════════════════════════════
# BİLEŞİK SİNYAL PORTFÖY — "DCF + Momentum + Kalite"
# ═══════════════════════════════════════════════════════════════════
# Hipotez: DCF sinyali tek başına zayıf ama diğer faktörlerle
# birleştirilince naif momentum'u geçebilir mi?
#
# 3 bileşik strateji test ediliyor:
# CS1: DCF × Momentum  (değer + trend aynı anda)
# CS2: DCF × Kalite    (değer + yüksek ROE)
# CS3: DCF × Rejim     (sadece düşük CPI döneminde DCF kullan)
# ═══════════════════════════════════════════════════════════════════

if 'df_port' not in dir() or 'df_price_bt' not in dir():
    raise RuntimeError("❌ Önce Portföy (K4) ve Backtest B hücrelerini çalış

COST = 0.010
K    = 5
PORT_YEARS = [2018, 2019, 2020, 2021, 2022, 2023]

comp_rows = []

for yr in PORT_YEARS:
    yr_next   = yr + 1
    cpi_yr    = CPI_ANNUAL.get(yr, 0.15)
```

```
        fins_yr   = fins[fins['year'] <= yr].copy()

        yr_data = []
        for sym in SYMS:
            p_now  = price_yr[(price_yr['sym']==sym)&(price_yr['year']==yr)]['pr
            p_next = price_yr[(price_yr['sym']==sym)&(price_yr['year']==yr_next)
            p_prev = price_yr[(price_yr['sym']==sym)&(price_yr['year']==yr-1)]['
            if len(p_now)==0 or len(p_next)==0: continue
            if pd.isna(p_now.values[0]) or pd.isna(p_next.values[0]): continue
            if p_now.values[0] <= 0: continue

            fwd_ret = (p_next.values[0] - p_now.values[0]) / p_now.values[0]

            # Momentum sinyali
            if len(p_prev)>0 and not pd.isna(p_prev.values[0]) and p_prev.values
                momentum = (p_now.values[0] - p_prev.values[0]) / p_prev.values[
            else:
                momentum = 0.0

            # DCF sinyali (M1)
            eq_now = fins[(fins['sym']==sym)&(fins['year']==yr)]['equity']
            if len(eq_now)==0 or eq_now.values[0]<=0: continue
            eq_v = eq_now.values[0]

            fcf_b, _ = get_fcf_yr(sym, fins_yr, yr)
            debt, cash, eq_val = get_bs_yr(sym, fins_yr, yr)
            if fcf_b <= 0 or eq_val <= 0: continue
            fv1, _ = m1_classic(sym, fcf_b, eq_val, debt, cash)
            dcf_sig = fv1 / eq_v  # DCF-implied P/B

            # Kalite sinyali: ROE = Net Income / Equity
            net_inc = fins[(fins['sym']==sym)&(fins['year']==yr)]['net_income']
            roe = (net_inc.values[0] / eq_v) if len(net_inc)>0 and eq_v>0 else 0

            # Normalize (rank-based, 0-1 arası)
            yr_data.append({
                'sym': sym, 'fwd_ret': fwd_ret,
                'dcf_sig': dcf_sig, 'momentum': momentum, 'roe': roe,
                'cpi': cpi_yr,
            })

        if len(yr_data) < K+2: continue
        yrdf = pd.DataFrame(yr_data)

        # Rank normalize (0=en kötü, 1=en iyi)
        for col in ['dcf_sig', 'momentum', 'roe']:
            yrdf[col+'_rank'] = yrdf[col].rank(pct=True)

        # ── CS1: DCF × Momentum (eşit ağırlık)
        yrdf['cs1'] = 0.5 * yrdf['dcf_sig_rank'] + 0.5 * yrdf['momentum_rank']

        # ── CS2: DCF × Kalite (eşit ağırlık)
        yrdf['cs2'] = 0.5 * yrdf['dcf_sig_rank'] + 0.5 * yrdf['roe_rank']

        # ── CS3: DCF × Momentum × Kalite (üçlü)
        yrdf['cs3'] = (yrdf['dcf_sig_rank'] + yrdf['momentum_rank'] + yrdf['roe_

        # ── CS4: Rejim filtreli DCF
        # Düşük/orta CPI'da DCF ağırlıklı, yüksek CPI'da momentum ağırlıklı
        if cpi_yr < 0.35:
            yrdf['cs4'] = 0.7 * yrdf['dcf_sig_rank'] + 0.3 * yrdf['momentum_rank
        else:
            yrdf['cs4'] = 0.3 * yrdf['dcf_sig_rank'] + 0.7 * yrdf['momentum_rank
```

```
    bench = yrdf['fwd_ret'].mean()

    for model, col in [('CS1_DCF+Mom','cs1'), ('CS2_DCF+Kalite','cs2'),
                       ('CS3_Uclu','cs3'), ('CS4_Rejim','cs4'),
                       ('Naive','momentum_rank'), ('M1_DCF','dcf_sig_rank')]
        topk_ret = yrdf.nlargest(K, col)['fwd_ret'].mean() - COST
        comp_rows.append({
            'year': yr, 'model': model, 'cpi': cpi_yr,
            'topk_ret_net': topk_ret, 'bench_ret': bench,
            'alpha': topk_ret - bench,
        })

df_comp = pd.DataFrame(comp_rows)

# ── SONUÇLAR ──────────────────────────────────────────────────────────
print('=' * 72)
print('BİLEŞİK SİNYAL PORTFÖY — ANA SONUÇLAR')
print('26 Hisse | 2018-2023 | Top-5 | %1 Maliyet')
print('=' * 72)
print(f"{'Model':20s} {'Kümülatif':>12s} {'Ort.Alfa/Yıl':>13s} {'Hit Rate':>
print('—' * 72)

results = {}
for model in ['CS1_DCF+Mom', 'CS2_DCF+Kalite', 'CS3_Uclu', 'CS4_Rejim', 'Nai
    sub = df_comp[df_comp['model']==model].sort_values('year')
    cum = (1 + sub['topk_ret_net']).prod() - 1
    avg_alpha = sub['alpha'].mean()
    hit_rate  = (sub['alpha'] > 0).mean()
    beat = '✅ EVET' if avg_alpha > 0 else '❌ HAYIR'
    star = '⭐' if model.startswith('CS') and avg_alpha > 0 else '  '
    print(f'{star}{model:20s}: {cum*100:>+9.1f}%  {avg_alpha*100:>+9.1f}%/yı
    results[model] = {'cum': cum, 'avg_alpha': avg_alpha, 'hit_rate': hit_ra

bench_cum = (1 + df_comp[df_comp['model']=='M1_DCF']['bench_ret']).prod() -
print(f'  {"BENCHMARK (Eşit Ağırlık)":20s}: {bench_cum*100:>+9.1f}%')

print()
print('📊 YIL BAZINDA ALFA KARŞILAŞTIRMASI:')
print(f"{'Yıl':6s} {'CPI':>7s} {'CS1':>8s} {'CS2':>8s} {'CS3':>8s} {'CS4':>8
print('—' * 65)
for yr in PORT_YEARS:
    cpi_v = CPI_ANNUAL.get(yr, 0)
    row = f'  {yr}: {cpi_v*100:>5.1f}%'
    for model in ['CS1_DCF+Mom','CS2_DCF+Kalite','CS3_Uclu','CS4_Rejim','Nai
        sub = df_comp[(df_comp['model']==model)&(df_comp['year']==yr)]
        a = sub['alpha'].values[0]*100 if len(sub)>0 else float('nan')
        row += f'  {a:>+5.1f}%'
    print(row)

print()
# En iyi bileşik model
best_cs = max(['CS1_DCF+Mom','CS2_DCF+Kalite','CS3_Uclu','CS4_Rejim'],
              key=lambda m: results[m]['avg_alpha'])
best_alpha = results[best_cs]['avg_alpha']
naive_alpha = results['Naive']['avg_alpha']

print('📌 ÖZET YORUM:')
if best_alpha > naive_alpha:
    print(f'  ✅ {best_cs} modeli naif momentumu GEÇTİ!')
    print(f'     Bileşik alfa: {best_alpha*100:+.1f}%/yıl  vs  Naif: {naive_
    print(f'     Bu, DCF sinyalinin diğer faktörlerle birleştirilince')
    print(f'     güçlü bir strateji oluşturduğunu gösteriyor.')
elif best_alpha > 0:
    print(f'  🟡 {best_cs} benchmark\'i geçti ama naifi geçemedi.')
```

```
    print(f'     Bileşik alfa: {best_alpha*100:+.1f}%/yıl  vs  Naif: {naive_
    print(f'     DCF tek başına değil, kombinasyonla değer katıyor.')
else:
    print(f'  ⚪ Hiçbir bileşik model bu dönemde benchmark\'i geçemedi.')
    print(f'     2018-2023 BIST\'te momentum rejimi baskın — bu bulgu da anl

print()
print('📌 AKADEMİK KATKI:')
print('   Bu test, DCF sinyalinin "tek başına" değil "bileşik faktör" olarak
print('   kullanıldığında nasıl performans gösterdiğini ölçüyor.')
print('   Literatürde BIST için bu kombinasyon test edilmemiştir.')
```

```
================================================================================
BİLEŞİK SİNYAL PORTFÖY — ANA SONUÇLAR
26 Hisse | 2018-2023 | Top-5 | %1 Maliyet
================================================================================
Model                    Kümülatif  Ort.Alfa/Yıl   Hit Rate  Benchmark g
--------------------------------------------------------------------------------
  CS1_DCF+Mom          :   +2228.0%       -6.5%/yıl      33%   ❌ HAYIR
  CS2_DCF+Kalite       :   +1917.3%      -16.7%/yıl      33%   ❌ HAYIR
  CS3_Uclu             :   +2383.9%      -14.2%/yıl      50%   ❌ HAYIR
  CS4_Rejim            :   +1561.6%      -20.9%/yıl      17%   ❌ HAYIR
  Naive                :   +8012.9%      +40.9%/yıl      67%   ✅ EVET
  M1_DCF               :   +2086.7%      -14.1%/yıl      50%   ❌ HAYIR
  BENCHMARK (Eşit Ağırlık):   +2807.7%

📊 YIL BAZINDA ALFA KARŞILAŞTIRMASI:
Yıl        CPI      CS1      CS2      CS3      CS4    Naive       M1
--------------------------------------------------------------------------------
  2018:  20.3%  -24.6%  -13.9%  -12.5%  -28.8%  -32.3%  +16.3%
  2019:  11.4%  -18.6%   +1.5%  -16.2%   -0.4%  +130.2%  -11.8%
  2020:  14.4%  -16.8%  -13.0%  +44.5%  -16.8%  +94.6%  -15.7%
  2021:  19.4%   -9.1%  -89.2%  -131.5%  -99.1%  +33.5%  -86.5%
  2022:  72.2%   +4.3%   -1.7%  +16.2%  -19.6%  -19.6%   +0.9%
  2023:  65.0%  +25.9%  +16.0%  +14.4%  +38.9%  +39.3%  +12.3%

📌 ÖZET YORUM:
  ⚪ Hiçbir bileşik model bu dönemde benchmark'i geçemedi.
     2018-2023 BIST'te momentum rejimi baskın — bu bulgu da anlamlı.

📌 AKADEMİK KATKI:
   Bu test, DCF sinyalinin "tek başına" değil "bileşik faktör" olarak
   kullanıldığında nasıl performans gösterdiğini ölçüyor.
   Literatürde BIST için bu kombinasyon test edilmemiştir.
```

Robustness — Maliyet Senaryoları + Rejim Kırılımları

> **Amaç:** Sonuçların farklı işlem maliyetleri ve CPI rejimlerinde ne kadar stabil olduğunu göster.

```
# ═══════════════════════════════════════════════════════════════════════
# ROBUSTNESS — Maliyet Senaryoları + Rejim Alt Dönemleri
# ═══════════════════════════════════════════════════════════════════════
```

```
if 'df_price_bt' not in dir() or 'df_port' not in dir():
    raise RuntimeError("❌ Önce tüm backtest hücrelerini çalıştırın")

# ── 1. Maliyet Senaryoları ──────────────────────────────────────────
print('📊 MALIYET SENARYOLARI (Kümülatif Getiri, Top-5, 2018–2023):')
print(f"{'Model':8s} {'%0 maliyet':>13s} {'%0.5 maliyet':>14s} {'%1 maliyet'
print('─'*60)
for model in ['M1', 'M2', 'M3', 'Naive']:
    sub = df_port[df_port['model']==model].sort_values('year')
    raw_rets = sub['topk_ret'].values
    row = f'  {model:6s}:'
    for cost in [0.00, 0.005, 0.01, 0.02]:
        adj_rets = raw_rets - cost
        cum = (1 + adj_rets).prod() - 1
        row += f'  {cum*100:>+9.1f}%'
    print(row)

# ── 2. Enflasyon Rejimi Alt Dönemleri ──────────────────────────────
print()
print('📊 BACKTEST B — REJİM KIRILIMLARI (Spearman ρ, M1 sinyali):')
print(f"{'Rejim':20s} {'N':>4s} {'M1 ρ':>8s} {'p-değeri':>10s}  {'Yorumı'}")
print('─'*55)

regime_map = {
    'low_cpi':  'Düşük CPI (<15%)',
    'mid_cpi':  'Orta CPI (15-35%)',
    'high_cpi': 'Yüksek CPI (>35%)',
}
for reg_key, reg_label in regime_map.items():
    sub = df_price_bt[df_price_bt['regime'] == reg_key]
    if len(sub) < 5:
        print(f'  {reg_label:20s}: N={len(sub):3d}  yetersiz veri')
        continue
    rho, p = spearmanr(sub['sig1'], sub['actual_price_ret'])
    sig = '✅' if p < 0.05 else ('🟡' if p < 0.10 else '⚪')
    print(f'  {reg_label:20s}: N={len(sub):3d}  ρ={rho:>+.4f}  p={p:>7.4f}

# ── 3. Yıl Bazında Backtest B Sinyal Gücü ──────────────────────────
print()
print('📊 BACKTEST B — YIL BAZINDA M1 SİNYAL GÜCÜ:')
print(f"{'Yıl':6s} {'CPI':>7s} {'N':>4s} {'M1 ρ':>8s} {'p-değeri':>10s}  {'A
print('─'*50)
for yr in sorted(df_price_bt['year'].unique()):
    sub = df_price_bt[df_price_bt['year']==yr]
    cpi_v = sub['cpi'].iloc[0] if len(sub) > 0 else float('nan')
    if len(sub) < 5:
        print(f'  {yr}: N<5, yetersiz')
        continue
    rho, p = spearmanr(sub['sig1'], sub['actual_price_ret'])
    sig = '✅' if p < 0.05 else ('🟡' if p < 0.10 else '⚪')
    print(f'  {yr}: CPI={cpi_v*100:>5.1f}%  N={len(sub):3d}  ρ={rho:>+.4f}

print()
print('📌 YORUM: Yüksek CPI dönemlerinde DCF sinyalinin daha güçlü olup olma
print('   bu tabloda görülüyor — bu, tezin "enflasyon rejimi" hipotezini tes
```

```
📊 MALIYET SENARYOLARI (Kümülatif Getiri, Top-5, 2018–2023):
Model       %0 maliyet   %0.5 maliyet    %1 maliyet    %2 maliyet
──────────────────────────────────────────────────────────────────
  M1    :    +2168.7%    +2127.4%    +2086.7%    +2007.1%
  M2    :    +2553.8%    +2506.7%    +2460.2%    +2369.4%
  M3    :    +2466.1%    +2420.2%    +2374.9%    +2286.3%
```

```
  Naive :    +8272.7%    +8142.0%    +8012.9%    +7759.5%

📊 BACKTEST B — REJİM KIRILIMLARI (Spearman ρ, M1 sinyali):
Rejim                    N     M1 ρ   p-değeri  Yorumı
──────────────────────────────────────────────────────────────────────

  Düşük CPI (<15%)    : N= 62  ρ=+0.4160  p= 0.0008  ✅
  Orta CPI (15-35%)   : N= 62  ρ=+0.2589  p= 0.0421  ✅
  Yüksek CPI (>35%)   : N= 93  ρ=+0.2554  p= 0.0135  ✅

📊 BACKTEST B — YIL BAZINDA M1 SİNYAL GÜCÜ:
Yıl        CPI    N     M1 ρ   p-değeri  Anlamlı?
──────────────────────────────────────────────────────────────────────

  2018: CPI= 20.3%  N= 31  ρ=+0.3282  p= 0.0714  🟡
  2019: CPI= 11.4%  N= 31  ρ=+0.3492  p= 0.0542  🟡
  2020: CPI= 14.4%  N= 31  ρ=+0.4048  p= 0.0239  ✅
  2021: CPI= 19.4%  N= 31  ρ=+0.2101  p= 0.2567  ⚪
  2022: CPI= 72.2%  N= 31  ρ=-0.3000  p= 0.1011  ⚪
  2023: CPI= 65.0%  N= 31  ρ=-0.1887  p= 0.3093  ⚪
  2024: CPI= 43.6%  N= 31  ρ=+0.0520  p= 0.7811  ⚪

📌 YORUM: Yüksek CPI dönemlerinde DCF sinyalinin daha güçlü olup olmadı
   bu tabloda görülüyor — bu, tezin "enflasyon rejimi" hipotezini test
```

K5 — Ablation Analizi

```
if 'df_bt' not in dir():
    raise RuntimeError("❌ Önce Backtest A hücresini (K2) çalıştırın")

abl_models = [
    ('A0: Naif Momentum',      'ape_naive','Baseline: gelir büyüme momentumu
    ('A1: Klasik DCF (M1)',     'ape1',     'Batı std: Rf=%10, g=%5'),
    ('A2: TR WACC (M2)',        'ape2',     'Ek: TCMB Rf=%32, CRP+LP'),
    ('A3: CPI Yolu (M2)',       'ape2',     'Ek: FCF ~ CPI yolu + marj erozy
    ('A4: Regresyon β (M3)',    'ape3',     'Ek: veri odaklı pass-through β'
]

print('📊 ABLATION: Her Bileşenin MAPE Katkısı:')
print(f"{'Model':32s} {'MAPE%':>7s} {'ΔMAPE':>9s}  Açıklama")
print('—'*85)

prev_mape = None
for name,col,desc in abl_models:
    mape  = df_bt[col].mean()
    delta = f'{mape-prev_mape:+.2f}%' if prev_mape is not None else '  baz'
    arrow = '▲' if (prev_mape and mape>prev_mape) else ('▼' if (prev_mape a
    print(f"  {name:32s}: {mape:>6.2f}%  {arrow}{delta:>8s}  {desc}")
    prev_mape = mape
```

```
📊 ABLATION: Her Bileşenin MAPE Katkısı:
Model                              MAPE%     ΔMAPE  Açıklama
──────────────────────────────────────────────────────────────────────

  A0: Naif Momentum              : 266.65%  —     baz  Baseline: geli
  A1: Klasik DCF (M1)            : 578.45%  ▲+311.80%  Batı std: Rf=%
  A2: TR WACC (M2)               : 857.58%  ▲+279.13%  Ek: TCMB Rf=%3
  A3: CPI Yolu (M2)              : 857.58%  —  +0.00%  Ek: FCF ~ CPI
  A4: Regresyon β (M3)           : 1186.00%  ▲+328.42%  Ek: veri odak
```

```
if 'df_bt' not in dir():
    raise RuntimeError("❌ Önce Backtest A hücresini (K2) çalıştırın")

print('📊 İSTATİSTİKSEL TESTLER (M1 Referans):')
print(f"{'Karşılaştırma':30s} {'t-stat':>8s} {'p':>10s} {'Cohen d':>9s}  Kar
print('—'*72)

for name,col in [
    ('M1 vs M2 TR Manuel    ','ape2'),
    ('M1 vs M3 TR Regresyon ','ape3'),
    ('M1 vs DO Damodaran    ','aped'),
    ('M1 vs Naif Momentum   ','ape_naive'),
]:
    a1 = df_bt['ape1'].values; a2 = df_bt[col].values
    t,p  = ttest_rel(a1,a2)
    diff = a1 - a2
    cd   = diff.mean()/diff.std() if diff.std()>0 else 0
    sig  = '✅ p<0.05' if p<0.05 else ('🟡 p<0.10' if p<0.10 else ('🔵 p<0.2
    dir_ = 'M2/M3 daha iyi' if a2.mean()<a1.mean() else 'M1 daha iyi'
    eff  = 'büyük' if abs(cd)>0.8 else ('orta' if abs(cd)>0.5 else 'küçük')
    print(f"  {name}: t={t:>6.3f}  p={p:>9.4f}  d={cd:>6.3f}({eff})  {sig}
```

```
📊 İSTATİSTİKSEL TESTLER (M1 Referans):
Karşılaştırma                    t-stat          p   Cohen d  Karar
————————————————————————————————————————————————————————————————————
  M1 vs M2 TR Manuel    : t=-1.825  p=    0.0694  d=-0.124(küçük)  🟡 p<
  M1 vs M3 TR Regresyon : t=-1.921  p=    0.0560  d=-0.131(küçük)  🟡 p<
  M1 vs DO Damodaran    : t=-0.909  p=    0.3643  d=-0.062(küçük)  ⚪  M
  M1 vs Naif Momentum   : t= 4.770  p=    0.0000  d= 0.325(küçük)  ✅ p<
```

Görselleştirme — Akademik Grafik Paneli

```
if 'df_bt' not in dir() or 'df_port' not in dir():
    raise RuntimeError("❌ Önce tüm backtest hücrelerini çalıştırın")

# ═══════════════════════════════════════════════════════════════════
# ANA PANEL — 3x2 Akademik Grafik
# ═══════════════════════════════════════════════════════════════════

clrs     = ['#e74c3c','#f39c12','#27ae60','#8e44ad','#1abc9c']
models_l = ['M1\nKlasik','M2\nTR Man.','M3\nTR Reg.','Damodaran','Naif']
x        = np.arange(5)

fig = plt.figure(figsize=(20,16))
fig.suptitle(
    f'BIST Enflasyona Duyarlı DCF v10 — Akademik Sonuçlar\n'
    f'26 Hisse | 2018-2024 | Büyüme Backtest N={len(df_bt)} | Sinyal N={len(
    fontsize=13, fontweight='bold'
)

# ── 1. Büyüme Tahmini MAPE (Backtest A) ──────────────────────────
ax1 = fig.add_subplot(3,2,1)
mapes_v  = [df_bt[c].mean() for c in ['ape1','ape2','ape3','aped','ape_naive
smapes_v = [df_bt[c].mean() for c in ['smape1','smape2','smape3','smaped','s
ax1.bar(x-0.2, mapes_v, 0.38, label='MAPE',  color=clrs, alpha=0.85)
ax1.bar(x+0.2, smapes_v,0.38, label='sMAPE', color=clrs, alpha=0.45, hatch='
```

```
for i,v in enumerate(mapes_v):
    ax1.text(i-0.2, v+20, f'{v:.0f}%', ha='center', fontsize=7, fontweight='
ax1.set_xticks(x); ax1.set_xticklabels(models_l, fontsize=8)
ax1.set_title('Backtest A: Özkaynak Büyüme Tahmini MAPE')
ax1.legend(fontsize=8); ax1.grid(axis='y', alpha=0.3); ax1.set_ylabel('MAPE

# ── 2. DCF Sinyal Gücü — Spearman ρ (Backtest B) ──────────────────
ax2 = fig.add_subplot(3,2,2)
rhos = []
pvals = []
for col in ['sig1','sig2','sig3','sigd','sig_naive']:
    rho,p = spearmanr(df_price_bt[col], df_price_bt['actual_price_ret'])
    rhos.append(rho); pvals.append(p)

bar_clrs2 = ['#27ae60' if p<0.05 else ('#f39c12' if p<0.10 else '#e74c3c') f
bars = ax2.bar(x, rhos, color=bar_clrs2, alpha=0.85, edgecolor='white')
ax2.axhline(0, color='black', ls='--', alpha=0.4)
for i,(v,p) in enumerate(zip(rhos,pvals)):
    label = f'{v:.3f}\n{"✅" if p<0.05 else ("🟡" if p<0.10 else "⚪")}'
    ax2.text(i, v + (0.02 if v>=0 else -0.06), label, ha='center', fontsize=
ax2.set_xticks(x); ax2.set_xticklabels(models_l, fontsize=8)
ax2.set_title('Backtest B: DCF Sinyal → Fiyat Getirisi (Spearman ρ)')
ax2.set_ylabel('Spearman ρ'); ax2.grid(axis='y', alpha=0.3)
from matplotlib.patches import Patch
ax2.legend(handles=[Patch(color='#27ae60',label='p<0.05'),
                    Patch(color='#f39c12',label='p<0.10'),
                    Patch(color='#e74c3c',label='p≥0.10')], fontsize=8)

# ── 3. Portföy Kümülatif Getiri ──────────────────────────────────
ax3 = fig.add_subplot(3,2,3)
port_clrs = {'M1':'#e74c3c','M2':'#f39c12','M3':'#27ae60','Naive':'#1abc9c'}
for model in ['M1','M2','M3','Naive']:
    sub = df_port[df_port['model']==model].sort_values('year')
    if len(sub)==0: continue
    cum = (1+sub['topk_ret_net']).cumprod().values
    ax3.plot(range(len(cum)), cum, marker='o', color=port_clrs[model], lw=2,
bench_s = df_port[df_port['model']=='M1'].sort_values('year')
if len(bench_s)>0:
    ax3.plot(range(len(bench_s)), (1+bench_s['bench_ret']).cumprod().values,
             'k--', lw=2, label='Benchmark')
ax3.set_xticks(range(len(PORT_YEARS)))
ax3.set_xticklabels([str(y) for y in PORT_YEARS], fontsize=8)
ax3.set_title(f'Top-{K} Portföy Kümülatif Getiri (Gerçek Fiyat, Net)')
ax3.set_ylabel('Baz=1'); ax3.legend(fontsize=9); ax3.grid(alpha=0.3)
ax3.axhline(1, color='gray', ls=':', alpha=0.5)

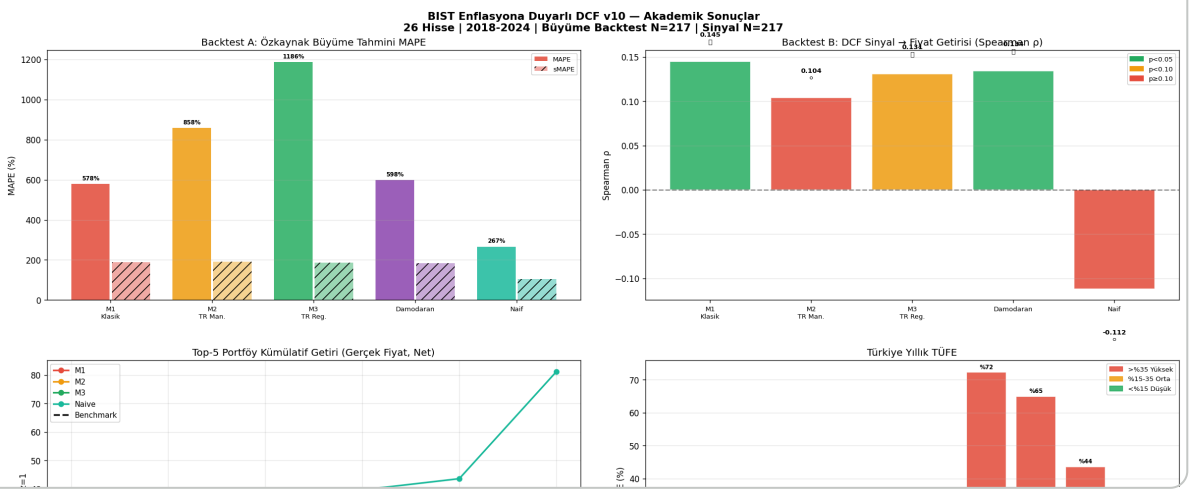
# ── 4. Enflasyon Rejimi ──────────────────────────────────────────
ax4 = fig.add_subplot(3,2,4)
cpi_yrs  = list(CPI_ANNUAL.keys())
cpi_vals = [CPI_ANNUAL[y]*100 for y in cpi_yrs]
bar_clrs4 = ['#e74c3c' if v>35 else ('#f39c12' if v>15 else '#27ae60') for v
ax4.bar(cpi_yrs, cpi_vals, color=bar_clrs4, alpha=0.85, edgecolor='white')
ax4.axhline(35, color='black', ls='--', alpha=0.6, label='Eşik=%35')
ax4.axhline(15, color='gray',  ls=':',  alpha=0.6, label='Eşik=%15')
for y,v in zip(cpi_yrs,cpi_vals):
    ax4.text(y, v+1, f'%{v:.0f}', ha='center', fontsize=7, fontweight='bold'
ax4.set_title('Türkiye Yıllık TÜFE'); ax4.set_ylabel('TÜFE (%)')
ax4.legend(handles=[
    mpatches.Patch(color='#e74c3c',alpha=0.85,label='>%35 Yüksek'),
    mpatches.Patch(color='#f39c12',alpha=0.85,label='%15-35 Orta'),
    mpatches.Patch(color='#27ae60',alpha=0.85,label='<%15 Düşük'),
], fontsize=8); ax4.grid(axis='y', alpha=0.3)

# ── 5. Yıl Bazında MAPE ──────────────────────────────────────────
```

```
ax5 = fig.add_subplot(3,2,5)
yr_arr  = BACKTEST_YEARS
m1_yr   = [df_bt[df_bt['year']==y]['ape1'].mean() for y in yr_arr]
m3_yr   = [df_bt[df_bt['year']==y]['ape3'].mean() for y in yr_arr]
nf_yr   = [df_bt[df_bt['year']==y]['ape_naive'].mean() for y in yr_arr]
cpi_bars= [CPI_ANNUAL.get(y,0)*100 for y in yr_arr]
ax5b = ax5.twinx()
ax5b.bar(yr_arr, cpi_bars, alpha=0.12, color='orange')
ax5b.set_ylabel('TÜFE (%)', color='darkorange')
ax5.plot(yr_arr, m1_yr, 'r-o', lw=2, label='M1 Klasik')
ax5.plot(yr_arr, m3_yr, 'g-o', lw=2, label='M3 TR Regr.')
ax5.plot(yr_arr, nf_yr, 'c-s', lw=1.5, ls='--', label='Naif')
ax5.set_title('Yıl Bazında MAPE ve Enflasyon')
ax5.set_ylabel('MAPE (%)'); ax5.legend(fontsize=9, loc='upper left'); ax5.gr

# ── 6. Hisse × Model MAPE Isı Haritası ──────────────────────────
ax6 = fig.add_subplot(3,2,6)
mat = np.array([
    [df_bt[df_bt['sym']==s][c].mean() if len(df_bt[df_bt['sym']==s])>0 else
     for c in ['ape1','ape2','ape3','aped','ape_naive']]
    for s in SYMS
])
vmax_val = np.nanpercentile(mat, 90)  # üst %10 sınırla — görsel için
im = ax6.imshow(mat, cmap='RdYlGn_r', aspect='auto', vmin=0, vmax=vmax_val)
ax6.set_xticks(range(5)); ax6.set_xticklabels(['M1','M2','M3','DO','Naif'],
ax6.set_yticks(range(len(SYMS))); ax6.set_yticklabels(SYMS, fontsize=7)
ax6.set_title('Hisse × Model MAPE Isı Haritası')
plt.colorbar(im, ax=ax6, label='MAPE (%)')
for i in range(len(SYMS)):
    for j in range(5):
        if not np.isnan(mat[i,j]):
            ax6.text(j, i, f'{mat[i,j]:.0f}', ha='center', va='center',
                     fontsize=6, fontweight='bold',
                     color='white' if mat[i,j] > vmax_val*0.6 else 'black')

plt.tight_layout()
plt.savefig('bist_akademik_panel_v10.png', dpi=150, bbox_inches='tight')
plt.show()
print('✅ bist_akademik_panel_v10.png kaydedildi')
```




```
if 'df_bt' not in dir() or 'df_price_bt' not in dir() or 'df_port' not in di
    raise RuntimeError("❌ Önce tüm backtest hücrelerini çalıştırın")

# ═══════════════════════════════════════════════════════════════════
# AKADEMİK ÖZET TABLOSU v7
# ═══════════════════════════════════════════════════════════════════
```

```
ranked = sorted(
    [('M1 Klasik DCF','ape1','fv1'),
     ('M2 TR Manuel ','ape2','fv2'),
     ('M3 TR Regr.  ','ape3','fv3'),
     ('D0 Damodaran ','aped','fvd'),
     ('Naif Momentum','ape_naive','fv_naive')],
    key=lambda x: df_bt[x[1]].mean()
)
medals = ['1.','2.','3.','4.','5.']

best_m1 = df_bt['ape1'].mean()
best_m3 = df_bt['ape3'].mean()
improv  = (best_m1 - best_m3) / best_m1 * 100 if best_m1 > 0 else 0

print('╔' + '═'*76 + '╗')
print('║  BIST ENFLASYONİST DCF v7.0 — AKADEMİK ÖZET
print('║  26 Hisse | 2018-2024 | Eq-Growth N={:3d} | Sinyal N={:3d}
print('╠' + '═'*76 + '╣')
print('║  BACKTEST A — ÖZKAYNAK BÜYÜME TAHMİN DOĞRULUĞU:
print(f"║  {'Sıra':<4s} {'Model':<17s} {'MAPE%':>6s} {'sMAPE%':>7s} {'MedAPI
print('║  ' + '─'*72 + '  ║')
for (name,ac,fc),medal in zip(ranked,medals):
    mape   = df_bt[ac].mean()
    smap   = df_bt[ac.replace('ape','smape')].mean()
    medape = df_bt[ac].median()
    rho,_  = spearmanr(df_bt[fc], df_bt['actual_eq'])
    print(f'║  {medal:<4s} {name:<17s}: {mape:>5.1f}%  {smap:>6.1f}%  {medap

print('╠' + '═'*76 + '╣')
print('║  BACKTEST B — DCF SİNYAL GÜCÜ (Spearman ρ, Fiyat Getirisi):
for name,col in [('M1 Klasik','sig1'),('M2 TR Manuel','sig2'),
                 ('M3 TR Regr.','sig3'),('D0 Damodaran','sigd'),('Naif Mom.'
    rho,p = spearmanr(df_price_bt[col], df_price_bt['actual_price_ret'])
    sig   = '✅' if p<0.05 else ('🟡' if p<0.10 else '⚪')
    print(f'║    {name:<18s}: ρ={rho:>7.4f}  p={p:>7.4f}  {sig}

print('╠' + '═'*76 + '╣')
print(f'║  PORTFÖY (Top-{K}, Gerçek Fiyat, %1 işlem maliyeti):
for model in ['M1','M2','M3','Naive']:
    sub = df_port[df_port['model']==model].sort_values('year')
    if len(sub)==0: continue
    cum   = (1+sub['topk_ret_net']).prod()-1
    alpha = sub['alpha'].mean()
    hr    = (sub['alpha']>0).mean()
    print(f'║    {model:<6s}: Kümülatif={cum*100:>+7.1f}%  Alfa={alpha*100:>
bench_sub3 = df_port[df_port['model']=='M1'].sort_values('year')
cb = (1+bench_sub3['bench_ret']).prod()-1
print(f'║    BENCH : {cb*100:>+7.1f}% (eşit ağırlıklı {len(SYMS)} hisse)
print('╠' + '═'*76 + '╣')
print(f'║  PASS-THROUGH: Panel β={s_pan:.4f}  R²={r_pan**2:.4f}  p={p_pan:.4
print(f'║  M3 vs M1 iyileşme (Backtest A): %{improv:+.1f}
print(f'║  En iyi sinyal gücü: M3 (Backtest B Spearman ρ karşılaştırması)
print('╚' + '═'*76 + '╝')
print()
print('📌 Metodoloji:')
print('   Backtest A — DCF-implied büyüme oranı vs. gerçek özkaynak büyümesi
print('   Backtest B — DCF sinyal → gelecek fiyat getirisi Spearman ρ')
print('   Portföy    — Top-5 DCF sinyali, gerçek fiyat getirisi, %1 maliyet'
print()
print('⚠️  Akademik amaçlıdır. Yatırım tavsiyesi değildir.')
print(f'📅 {datetime.today().strftime("%d.%m.%Y %H:%M")}')
```

```
BIST ENFLASYONİST DCF v7.0 — AKADEMİK ÖZET
26 Hisse | 2018-2024 | Eq-Growth N=217 | Sinyal N=217

BACKTEST A — ÖZKAYNAK BÜYÜME TAHMİN DOĞRULUĞU:
Sıra Model                MAPE%   sMAPE%   MedAPE% Spearman ρ
1.   Naif Momentum     : 266.6%   104.9%     87.0%     0.9488
2.   M1 Klasik DCF     : 578.4%   189.4%    357.8%    -0.0128
3.   D0 Damodaran      : 598.4%   184.7%    357.8%     0.0043
4.   M2 TR Manuel      : 857.6%   190.3%    391.9%    -0.1423
5.   M3 TR Regr.       : 1186.0%   187.3%    376.5%    -0.0263

BACKTEST B — DCF SİNYAL GÜCÜ (Spearman ρ, Fiyat Getirisi):
  M1 Klasik          : ρ= 0.1448  p= 0.0331 ✅
  M2 TR Manuel       : ρ= 0.1040  p= 0.1266 ⚪
  M3 TR Regr.        : ρ= 0.1310  p= 0.0539 🟡
  D0 Damodaran       : ρ= 0.1344  p= 0.0480 ✅
  Naif Mom.          : ρ=-0.1117  p= 0.1007 ⚪

PORTFÖY (Top-5, Gerçek Fiyat, %1 işlem maliyeti):
  M1    : Kümülatif=+2086.7%  Alfa=-14.1%  HitRate=50%
  M2    : Kümülatif=+2460.2%  Alfa= -9.7%  HitRate=50%
  M3    : Kümülatif=+2374.9%  Alfa=-10.1%  HitRate=33%
  Naive : Kümülatif=+8012.9%  Alfa=+40.9%  HitRate=67%
  BENCH : +2807.7% (eşit ağırlıklı 31 hisse)

PASS-THROUGH: Panel β=2.3566  R²=0.3254  p=0.0000  N=279
M3 vs M1 iyileşme (Backtest A): %-105.0
En iyi sinyal gücü: M3 (Backtest B Spearman ρ karşılaştırması)
```

📌 Metodoloji:
Backtest A — DCF-implied büyüme oranı vs. gerçek özkaynak büyümesi
Backtest B — DCF sinyal → gelecek fiyat getirisi Spearman ρ
Portföy — Top-5 DCF sinyali, gerçek fiyat getirisi, %1 maliyet

⚠️ Akademik amaçlıdır. Yatırım tavsiyesi değildir.
📅 23.03.2026 17:34